PROJECT BASED LEARNING (PBL) REPORT
ON

SMART KRISHI

Precision Agriculture and Decision Support Platform for Indian Farmers

Submitted in partial fulfilment of the requirement for the award of Degree of Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Ravina	**[24EARCS131]**
Shrashti Jain	**[24EARCS154]**
Venkatesh Kumawat	**[24EARCS181]**
Yashvardhan Singhal	**[24EARCS188]**
Vinay Choudhary	**[24EARCS184]**

Under the supervision of

PROJECT GUIDE
Er. Ram Babu Buri
(Assistant Professor)

PROJECT COORDINATOR
Dr. Vishal Shrivastava
Professor

Academic Year 2026-27 | October 2026

Department of Computer Science & Engineering
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC | NBA

Department of Computer Science and Engineering

Certificate

This is to certify that the project report entitled "Smart Krishi: Precision Agriculture and Decision Support Platform for Indian Farmers" is a bonafide record of the Project Based Learning (PBL) work carried out by the team members listed on the title page, under my guidance, in partial fulfilment of the requirements of the semester curriculum of the Department of Computer Science Engineering , Arya College of Engineering & Information Technology , during the academic year 2026-27.

The work embodied in this report has not been submitted elsewhere for the award of any degree or diploma.

Mentor
Er.Ram Babu Buri
Assistant Professor

HOD
Dr.Akhil Pandey
Professor

Department of Computer Science and Engineering

Declaration

We **Ravina[24EARCS131], Shrashti Jain[24EARCS154], Yashvardhan[24EARCS187] Venkatesh Kumawat [24EARCS181] and Vinay Choudhary [24EARCS184]** hereby declare that this project report titled "Smart Krishi" is the result of our own work, carried out as a team during the PBL semester. Wherever the work of others has been used, including published research, government data sources and open-source libraries, it has been acknowledged in the text and in the list of references. The source code, test results and screenshots presented in this report were produced by the team.

We have followed the academic integrity guidelines of the institution and have not fabricated any data. In line with the design principles of the system itself, no weather, price or agronomic value is simulated or invented in this work.

Team Smart Krishi

Ravina [24EARCS131]
Shrashti Jain[24ERCS154]
Venkatesh Kumawat[24EARCS181]
Yashvardhan Singhal[24EARCS188]
Vinay Choudhary[24EARCS184]

Acknowledgement

We express our sincere gratitude to our project mentor, Er.Ram Babu Buri, for continuous guidance, timely feedback and for encouraging us to ground the project in published research. We thank the Head of the Department and the faculty of Computer Science And Engineering for providing the resources and the environment needed to complete this work.

We acknowledge the open data and documentation provided by the Open-Meteo project and by the Ministry of Agriculture and Farmers Welfare through data.gov.in (AGMARKNET), and the guidance published by the Indian Council of Agricultural Research (ICAR), without which a verified, data-honest platform would not have been possible. We also thank the open-source communities behind Spring Boot, Spring Security, Hibernate, Bootstrap and JUnit.

Finally, we thank our families and friends for their support throughout the ten weeks of this project.

Abstract

Indian smallholder farmers take decisions on crop selection, irrigation, fertilizer use, costing and selling with little verified, location-specific information. Existing agricultural applications usually address a single decision and many rely on static or fabricated data. Smart Krishi is an integrated, rule-based precision agriculture platform that connects these decisions for the same farmer and farm plots, and that presents only verified outputs.

The platform comprises nine modules: farmer management and land profiling; multi-parameter crop recommendation; live weather and evapotranspiration-informed irrigation advisory using Open-Meteo; ICAR-standard fertilizer advice converted into commercial bag quantities with soil pH amendments; profit, ROI and break-even analysis; a six-stage crop calendar; short-term price trend prediction from moving averages and linear momentum; a Haversine-based nearby mandi locator with live data.gov.in AGMARKNET prices and a transparent database fallback; and an administration portal.

The system is implemented as a stateless Java 17 / Spring Boot 3.3.4 backend secured with Spring Security 6 and JWT-based role-based access control, a MySQL 8.0 (InnoDB) persistence layer with six normalised tables, and a lightweight HTML5, CSS3, ES6+ and Bootstrap 5.3 frontend. Sixteen automated JUnit 5 integration test cases covering security, external-API time-outs, agronomic safety boundaries and data failover were executed with zero failures and zero errors. This report documents the literature survey of 32 studies that motivate the system, the requirements, design, implementation and testing of each module, the deployment strategy, the project management record and the limitations and future work.

Table of Contents

1 Introduction 1

2 Literature Survey 2

3 Requirement Analysis 6

4 System Design 9

5 Implementation of Modules 14

5.1 Module 1: Farmer Management and Land Profiling 15
5.2 Module 2: Precision Crop Recommendation Engine 16
5.3 Module 3: Live Weather and Smart Irrigation Advisory 17
5.4 Module 4: Smart Fertilizer Recommendation Advisor 19
5.5 Module 5: Crop Profit and ROI Calculator 21
5.6 Module 6: Crop Growth Calendar and Phenological Advisory 22
5.7 Module 7: Crop Price Trend Prediction Engine 23
5.8 Module 8: Nearby APMC Mandi Prices and Geodesic Locator 24
5.9 Module 9: System Administration and Governance 26
5.10 Responsive Design 27

6 Security Design 27

7 Testing and Verification 29

8 Deployment and Configuration 31

9 Project Management 31

10 Results and Discussion 33

11 Limitations and Future Work 33

12 Conclusion 34

References 34

Appendix A: Screenshot and Code Capture Checklist 37

1 Introduction

1.1 Background

Agriculture remains the primary livelihood for a large share of India's population. Most of India's farmers are marginal or small holders who operate on fragmented plots, have limited access to soil testing and extension services, and sell into markets where price information is uneven [29], [30], [32]. The decisions that determine a season's income are taken early and are difficult to reverse: which crop to sow, how much water and fertilizer to apply, what the crop will cost and earn, and where and when to sell.

Digital advisory services can narrow this information gap. A voice-based advisory built trust and changed input choices among cotton farmers in Gujarat [27], and mobile advisories helped rice farmers in Odisha cope with weather shocks [28]. Yet adoption among smallholders is uneven [29]-[31], and most tools cover only one decision. A farmer who wants to choose a crop, check the weather, calculate fertilizer, estimate profit and find a market must currently consult several unconnected sources, some of which display unverified or outdated values.

1.2 Motivation

Three observations from the literature motivated this project. First, soil information is rarely turned into action: over 90% of farmers surveyed had never tested their soil before the Soil Health Card scheme, and even among card holders only 48% followed the recommended fertilizer rates [14], [R1]. Second, weather forecasts have documented economic value but must be credible and converted into specific advice [5], [7], [8]. Third, access to transport and, more strongly, to market information improves the prices farmers realise [23]. These findings point to a platform that is integrated, actionable and honest about its data.

1.3 Problem Statement

To design, implement and verify a web-based, mobile-responsive decision-support platform that (i) recommends suitable crops from soil and climate parameters, (ii) gives live, location-specific irrigation guidance, (iii) calculates fertilizer requirements in the physical units farmers actually purchase, (iv) projects the profitability of a crop before sowing, (v) indicates short-term price trends and nearby market prices, and (vi) does all of this with secure, role-based access and without ever presenting fabricated data.

1.4 Objectives

1. Provide secure farmer registration, authentication and multi-plot farm profiling using stateless JWT-based role-based access control.
2. Implement a rule-based crop recommendation engine that evaluates eight soil and climatic parameters and prescribes agronomically suitable crops.
3. Integrate live Open-Meteo weather data via browser GPS or city search and generate an irrigation decision (for example, PAUSE IRRIGATION when rainfall exceeds 5 mm).

4. Compute ICAR-standard nutrient deficits and convert them into commercial bag counts, with lime and gypsum recommendations for acidic and alkaline soils.
5. Provide a profit, ROI and break-even calculator that aggregates seven production cost heads.
6. Generate six-stage crop growth calendars and statistical short-term price predictions.
7. Locate nearby APMC mandis using the Haversine formula and show live government prices with a transparent fallback to verified data.
8. Provide an administration portal for auditing farmers, publishing schemes and seeding market prices.
9. Verify the platform with automated integration tests and ensure zero fabricated telemetry.

1.5 Scope and Exclusions

The platform covers nine modules as defined in the Software Requirements Specification (SRS v2.0) [R4]. The earlier AI plant-disease detection module was deliberately removed in the final milestone to eliminate dependence on third-party computer-vision APIs and the failures they introduced. The platform is a decision-support tool: it informs the farmer but does not execute transactions, process payments or replace laboratory soil testing and the advice of agricultural extension officers.

1.6 Methodology

The work followed a ten-week PBL schedule. Weeks 1 and 2 were spent on problem definition, roles and architecture; week 3 on UML design; week 4 on database design and UI mock-ups; weeks 5 to 8 on module development by role; week 9 on system integration and end-to-end testing; and week 10 on final evaluation, documentation and deployment preparation. An incremental, module-by-module approach was used so that each module could be developed, tested and demonstrated independently while sharing one identity and data layer. Chapter 9 records the weekly progress.

1.7 Organisation of the Report

Chapter 2 surveys the related literature. Chapter 3 presents the requirements. Chapter 4 describes the system and database design. Chapter 5 details the implementation of each module with algorithms and screenshots. Chapter 6 covers security. Chapter 7 reports testing. Chapter 8 describes deployment. Chapter 9 documents project management. Chapter 10 discusses results, Chapter 11 limitations and future work, and Chapter 12 concludes. The appendices contain the traceability matrix and a checklist of screenshots.

A survey of 32 research papers and 3 supporting reports was carried out across the problem areas addressed by the platform. The studies were located through web searches of journal repositories and institutional archives and are listed in the References; each should be confirmed on Google Scholar for its final citation details. This chapter summarises the findings by theme and then positions Smart Krishi against them.

2 Literature Survey

2.1 Crop Recommendation Systems

Machine-learning crop recommenders typically use nitrogen, phosphorus, potassium, pH, temperature, humidity and rainfall as inputs. Kathoon et al. apply neural network and CNN models [1]; a Madras Agricultural Journal study trains seven classifiers on the same variables and exposes them through a simple web form [2]; others use Random Forest and related ensembles on Indian crop data [3], [4]. The seven-variable input set is therefore the accepted standard. However, these systems depend on training datasets whose provenance and regional validity are often unclear, and they report classification accuracy rather than agronomic safety.

2.2 Weather-Based Irrigation and Advisory

Irrigation-Advisor derives irrigation recommendations from weather forecasts without on-site sensors and achieved water savings of up to 13% compared with farmer practice, with lettuce an exception [5]. Pelosi et al. forecast crop evapotranspiration with a median bias of 0.2 mm and an RMSE of 0.75 mm on the first forecast day [6]. In India, agromet advisory adopters showed better yield and yield attributes [7], and farmers were satisfied with advisories although forecast credibility needed refinement [8]. Open-Meteo provides free hourly and daily reference evapotranspiration (ET0) forecasts without an API key for non-commercial use [R3], which makes forecast-driven irrigation feasible for a low-cost platform.

2.3 Soil-Test-Based Fertilizer Recommendation

Declining soil health affects an estimated 120 million hectares, about 37% of India's area, and site-specific nutrient management depends on soil testing [10]. The Soil Test Crop Response (STCR) approach underpins decision-support tools [12], [13], and ICRISAT derived recommendations from 40,265 soil samples across 30 districts of Odisha [9]. Rule-based calculators such as FertiCal-P convert soil-test values into NPK advice [11]. The Soil Health Card programme found that over 90% of surveyed farmers had not tested their soil and 92% were not using balanced fertilizer [R1]. Awareness of the card was 82%, but only 66% understood it and 48% followed the recommended rates [14]. Where advice was followed, urea use fell without yield or financial loss [15], and adopters used about 20 kg/ha less nitrogen and 19 kg/ha less phosphorus [16]. The gap is the actionability of recommendations, not only their quality.

2.4 Profitability Analysis

Cost concepts A2, B2 and C2, cost of production (C2 divided by yield), net return and benefit-cost ratio are the standard instruments for assessing crop profitability [17]. Farmers lack a simple tool that applies these concepts to their own plot before sowing.

2.5 Price Forecasting and Market Access

AGMARKNET is the principal public price source. ARIMA has been applied to wheat prices, which exhibit marked seasonality [18], and to cotton prices in Haryana, where forecasts help farmers choose between nearby markets [19]. ARFIMAX uses market arrivals as an exogenous variable for mustard [20]; ARIMA, RNN, LSTM, VAR, Random Forest and XGBoost have been compared for onion, potato and tomato [21]; and an interpretable web forecaster serves small farmers using over a thousand markets [22]. On access,

transportation and especially market information improve the prices farmers receive [23]. Studies of e-NAM report mixed outcomes: limited price impact [24] and difficulties for farmers [25], while selling through mandis gave local-crop producers 13 to 73% higher prices than private traders [26].

2.6 Digital Advisory Adoption

Digital and precision agriculture in India face fragmented holdings, weak data infrastructure and unequal access [29], [30]. Service-based adoption outperforms individual technology ownership for smallholders [31], and one Chhattisgarh sample showed no adoption of precision technologies at all [32]. Advisory evaluations show trust and input adoption [27] and benefits for disadvantaged groups [28], but no systematic yield gains [27]. These findings argue for tools that are free, mobile-friendly, simple and trustworthy.

2.7 Summary Table of Reviewed Studies

Table 1: Reviewed studies and the Smart Krishi module each informs (n/a: not captured; confirm on Google Scholar)

#	Study	Year	Key contribution / finding	Module
1	Kathoon et al.	2025	Neural network / CNN crop recommender using N, P, K, pH, temperature, humidity and rainfall.	M2
2	AI-based Smart Crop Recommendation System (Madras Agric. J.)	2025	Trains seven ML models on the same seven inputs; farmer enters values through a simple web form.	M2
3	A Machine Learning-Enabled System for Crop Recommendation (Eng. Proc. 67)	n/a	ML crop recommendation built from weather, soil and NPK data; reviews earlier classifier and DSS work.	M2
4	A Reliable ML Framework for Crop Recommendation in Precision Agriculture	n/a	Random Forest / XGBoost-style classifiers on NPK, pH, humidity, temperature and rainfall for Indian crops.	M2
5	Miras-Ávalos et al. (Irrigation-Advisor)	2019-20	Forecast-driven irrigation DSS without on-site sensors; water savings up to 13% vs farmer practice (lettuce was an exception).	M3
6	Pelosi et al.	n/a	Forecasts crop evapotranspiration ahead of time (median bias 0.2 mm, RMSE 0.75 mm on day 1) to support irrigation advisories.	M3
7	Validation of medium-range weather forecasts (PAU)	n/a	Agromet advisory adopters showed better yield and yield attributes than non-adopters.	M3
8	Agromet advisory adoption, semi-arid southern India (Front. Sustain. Food Syst.)	2023	Farmers satisfied with agromet advisories, but forecast credibility needs improvement for sustained use.	M3
9	Chander et al. (ICRISAT)	2020	Soil-test-based fertilizer recommendations derived from 40,265 soil samples across 30 districts of Odisha.	M4
10	User-centered design of a fertilizer recommendation system (MIT thesis)	n/a	Mobile fertilizer recommendation engine for smallholders under site-specific nutrient management; notes ~120 Mha (37%) of Indian land affected by soil degradation.	M4
11	FertiCal-P Android DSS	n/a	Rule-based + regression NPK calculator from soil-test N, P2O5, K2O and pH.	M4
12	STCR & Dhartimitra, cowpea, Karnataka (Madras Agric. J.)	2025	STCR-based recommendations using decision-support software outperformed conventional recommendations.	M4

#	Study	Year	Key contribution / finding	Module
13	DSS for fertilizer recommendation: a case study (Dharwad)	2014	Targeted-yield DSS built on Soil Test Crop Response research for farmers' fertilizer decisions.	M4
14	The Soil Health Card scheme in India: lessons learned	n/a	Awareness 82%, but only 66% understood and only 48% followed recommended fertilizer rates.	M4
15	Impact of Soil Health Card on urea use, Haryana (IJEE)	n/a	Following card advice reduced urea use without lowering yields or causing financial loss.	M4
16	Socio-economic impact of Soil Health Card, Andhra Pradesh (IJAgS)	n/a	Adopters used ~20 kg/ha less N and ~19 kg/ha less P, reducing excess fertilizer in paddy.	M4
17	Profitability of coarse cereals production in India	n/a	Uses cost concepts A2/B2/C2, cost of production = C2 / yield, net return and benefit-cost ratio.	M5
18	Cariappa et al. (wheat price forecasting)	2020	ARIMA on AGMARKNET monthly prices; strong seasonality; stresses need to disseminate forecasts to farmers.	M7
19	Kumar et al. (cotton, Haryana)	2024	ARIMA(1,1,1) on AGMARKNET data; forecasts help farmers choose between nearby markets.	M7
20	Paul et al. (mustard, ARFIMAX)	2022	Long-memory model using arrivals as exogenous variable on daily Bharatpur and Agra prices.	M7
21	Suresha & Uppaladinni	2021	Compares ARIMA, RNN, LSTM, VAR, Random Forest and XGBoost for onion, potato, tomato prices.	M7
22	Interpretable produce price forecasting for small/marginal farmers	n/a	Pulls Agmarknet data for 1000+ markets and shows interpretable forecasts in a mobile-friendly web app.	M7
23	Negi, Birthal et al. (Food Policy)	n/a	Access to transport and market information improves price terms; the information effect is stronger.	M8
24	Venkatesh et al. (changing structure of agri marketing, e-NAM)	n/a	Studies e-NAM trade patterns, benefits and constraints; impact on farmers' prices reported as limited.	M8
25	Sai Shiva, Nair & Thajudeen (IIMK WP 491)	n/a	Finds e-NAM digitization created difficulties and did not secure remunerative prices.	M8
26	Agricultural marketing channels and market prices (Econ. Analysis & Policy 81)	2024	Local crop producers got 13-73% higher prices through mandis than private traders; export crops 19% less.	M8
27	Cole & Fernando (Economic Journal)	2021	Voice-based advisory built trust and input adoption among cotton farmers; no systematic yield gains found.	All
28	Harigaya & Zhu (Precision Development)	2025	Mobile advisory helped farmers cope with weather shocks; disadvantaged groups benefited substantially.	All
29	Shaheen et al.	2020	Reviews challenges and opportunities of precision farming in India with farmer interviews near Hyderabad.	All
30	Whither digital agriculture in India?	n/a	Review: fragmented holdings, weak data infrastructure and unequal digital access slow smallholder adoption.	All
31	Rajesh & Malarkodi	n/a	Finds integrative agronomy + economics evidence is scarce; service-based adoption beats individual ownership.	All
32	Sathyamala	n/a	Argues digital tools may not benefit marginal farmers; in a Chhattisgarh sample none had adopted precision agriculture.	All

2.8 Research Gap and Positioning

Table 2 contrasts representative existing systems with Smart Krishi. The surveyed systems each cover one decision. The research gap is an integrated, transparent and safe platform that spans the chain from pre-sowing decisions to sale, remains usable on low-cost devices, and refuses to display unverified data.

Table 2: Coverage of representative systems versus Smart Krishi

System	Crop	Irrigation	Fertilizer	Price	Mandi
Crop ML systems [1]-[4]	Yes (learned)	No	No	No	No
Irrigation-Advisor [5]	No	Yes	No	No	No
FertiCal-P, STCR DSS [11]-[13]	No	No	Yes	No	No
Price forecasting [18]-[22]	No	No	No	Yes	No
Smart Krishi (this work)	Yes (rules)	Yes	Yes (bags)	Basic trend	Yes (GPS)

3 Requirement Analysis

The requirements are specified in the Software Requirements Specification, version 2.0, prepared according to IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018 [R4]. Requirements are labelled FR-<Module>-<Number> for functional and NFR-<Category>-<Number> for non-functional requirements, with priorities HIGH, MEDIUM and LOW. This chapter summarises them.

3.1 User Classes

Table 3: User classes and characteristics

User class	Characteristics	Goals
Registered Farmer (ROLE_FARMER)	Primary user; farming background with basic digital literacy on mobile or desktop browsers	Maximise yield, save water and input cost, forecast profit, find the nearest high-paying APMC mandi
Platform Administrator (ROLE_ADMIN)	System controllers and agricultural officers	Monitor usage, curate verified commodity prices, post government welfare advisories, audit farmers
Public / Unauthenticated User	Farmers exploring the platform before registration	Use public tools: crop recommendation, weather, fertilizer advisor, profit calculator, calendar, price prediction, mandi prices

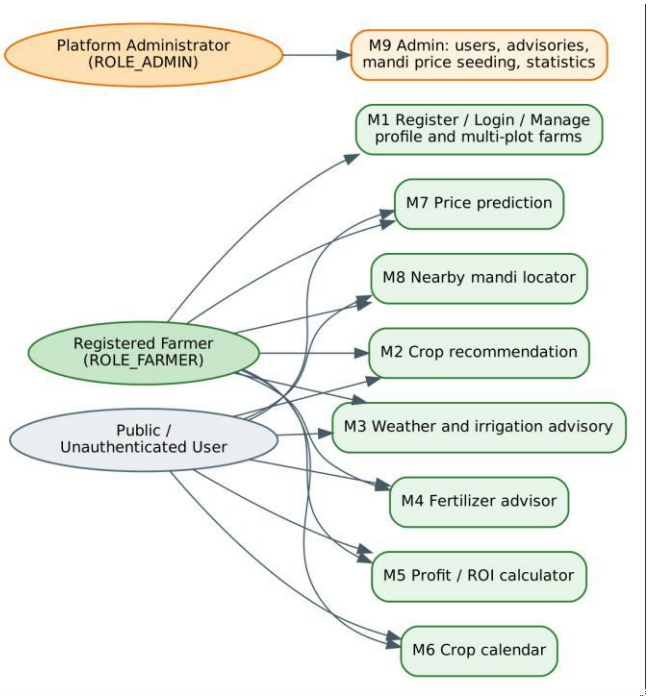


Figure 1: Use-case view: actors and the modules they can access

3.2 Operating Environment

- Client: modern HTML5 browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+) on desktop, tablet and smartphone.
- Backend: Java 17 LTS, Spring Boot 3.3.4, embedded Apache Tomcat 10.1, Linux or Windows, Docker containers.
- Database: H2 in-memory (MODE=MySQL) for development; MySQL Server 8.0+ with InnoDB and utf8mb4 for production.
- Hosting targets: Netlify for static frontend assets and Render for the Dockerised Spring Boot service.

3.3 Design and Implementation Constraints

1. Stateless authentication: no HTTP session; Spring Security validates JWT Bearer tokens.
2. Zero fake weather data: hard-coded, randomised or simulated weather is prohibited; invalid locations return an explicit 404.
3. Transparent data sourcing: the mandi service must disclose whether data is live, cached or from the verified fallback.
4. ICAR agronomic adherence: fertilizer grades are Urea 46% N, DAP 18:46:0, MOP 60% K2O and SSP 16% P2O5.
5. No AI disease-detection dependency: the module is permanently decommissioned.

3.4 Assumptions and Dependencies

- Open-Meteo APIs remain accessible without authentication for standard geocoding and forecast queries.
- Client devices have internet access and, where applicable, grant browser geolocation permission.
- Subsidised MRP rates for standard fertilizers remain within nominal statutory brackets (used only for approximate cost estimation).

3.5 Functional Requirements

Table 4: Summary of functional requirements from SRS v2.0

IDs	Module	Summary of requirements (priority)
FR-1.1 to 1.5	M1 Farmer management	Registration with name, email, phone, password (HIGH); BCrypt hashing, cost 10; JWT issuance valid 24 h (HIGH); profile with village, district, state, land area, soil, irrigation, primary crop (HIGH); multi-plot farms (MEDIUM); RBAC so farmers see only their own records (HIGH)
FR-2.1 to 2.4	M2 Crop recommendation	Validate N 0-300, P 0-200, K 0-300 kg/ha, pH 3-11, temperature -10 to 60 C, humidity 0-100%, rainfall 0-3000 mm and soil type (HIGH); match suitability matrices for ten crops (HIGH); return crop, season, growing conditions, fertilizer suggestion (MEDIUM); log recommendations (MEDIUM)
FR-3.1 to 3.5	M3 Weather and irrigation	Resolve city via Open-Meteo Geocoding, 404 if not found (HIGH); accept GPS coordinates (HIGH); return temperature, condition, humidity, wind speed, rainfall, timestamp (HIGH); irrigation decision model (HIGH); five-day forecast (MEDIUM)
FR-4.1 to 4.6	M4 Fertilizer advisor	Parse crop, soil, stage, area, N, P, K, pH (HIGH); deficit = max(0, target - soil) (HIGH); bag conversion (HIGH); split schedule (MEDIUM); lime / gypsum by pH (HIGH); cost estimate (MEDIUM)
FR-5.1 to 5.6	M5 Profit and ROI	Seven cost inputs (HIGH); gross revenue (HIGH); cost aggregation (HIGH); net profit, profit per acre, ROI (HIGH); break-even price (HIGH); financial health class (MEDIUM)
FR-6.1 to 6.4	M6 Crop calendar	Six phenological stages (HIGH); day ranges, irrigation and top-dressing milestones (MEDIUM); state-specific notices (MEDIUM); generic 120-day fallback (LOW)
FR-7.1 to 7.5	M7 Price prediction	History evaluation, minimum 3 points (HIGH); INSUFFICIENT_DATA handling (HIGH); SMA, momentum and forecast formula (HIGH); trend and confidence 50-95% (MEDIUM); risk disclaimer (LOW)
FR-8.1 to 8.7	M8 Mandi locator	Haversine distance (HIGH); radius 10/25/50/100 km (HIGH); manual state and district fallback (HIGH); sort nearest / price_high / price_low (HIGH); source tag (HIGH); Google Maps link (MEDIUM); service status (MEDIUM)
FR-9.1 to 9.4	M9 Administration	Statistics (HIGH); farmer directory and removal (HIGH); mandi price creation (MEDIUM); advisory publication, e.g. PM-KISAN, PMFBY, Soil Health Card (MEDIUM)

3.6 Non-Functional Requirements

Table 5: Non-functional requirements

ID	Category	Requirement
NFR-PERF-1	Response time	Calculation endpoints (fertilizer, profit, calendar, prediction) respond within 250 ms under normal load
NFR-PERF-2	External API latency	Open-Meteo queries time out after 5,000 ms and fail gracefully with user-friendly notices
NFR-PERF-3	Client footprint	Frontend page load under 1.5 s on a standard 4G connection
NFR-SAFE-1	Fertilizer safety	Enforce upper application thresholds to prevent soil toxicity, nitrogen burn and groundwater pollution
NFR-SAFE-2	Zero fake telemetry	Weather and mandi prices are never randomised or fabricated; explicit notices or verified fallbacks are shown
NFR-SEC-1	Stateless JWT	/api/farmers/** and /api/admin/** require a valid Bearer token
NFR-SEC-2	Hashing	Passwords salted and hashed with BCrypt; never stored or logged in plain text
NFR-SEC-3	Injection defence	All queries use Spring Data JPA parameterised statements
NFR-SEC-4	XSS prevention	Dynamic DOM injection uses textContent or pre-sanitised templates
Quality	Availability, reliability, maintainability, portability	99.5% uptime target; seamless AGMARKNET failover; layered architecture; standalone JAR / Docker deployable

3.7 External Interfaces

Table 6: External interfaces

Interface	Endpoint / mechanism	Usage
Open-Meteo Geocoding	https://geocoding-api.open-meteo.com/v1/search (name, count=1, language=en, format=json)	Resolve place names to latitude and longitude
Open-Meteo Forecast	https://api.open-meteo.com/v1/forecast (latitude, longitude, current_weather, hourly humidity and precipitation, daily weathercode and temperatures, timezone=auto)	Real-time temperature, wind, precipitation, humidity and five-day forecast
data.gov.in AGMARKNET	https://api.data.gov.in/resource/... (api-key, format=json, filters[state], filters[district])	Live commodity mandi rates from the Ministry of Agriculture and Farmers Welfare
Browser Geolocation	navigator.geolocation.getCurrentPosition()	Device GPS coordinates for weather and mandi search
Communications	HTTP/1.1 and HTTP/2 over TLS; JSON (application/json; charset=UTF-8); CORS for http://localhost:3000 and https://*.netlify.app	Client-server data exchange

4 System Design

4.1 Architecture Overview

Smart Krishi follows a distributed multi-tier client-server architecture. The browser client communicates with a stateless Spring Boot service tier through asynchronous JSON REST calls using the Fetch API. The service tier consists of Spring Security with a JWT filter, REST controllers, business-logic services with external API clients, and Spring Data JPA repositories that access the persistence tier. Outbound HTTP requests from the service tier reach Open-Meteo and data.gov.in.

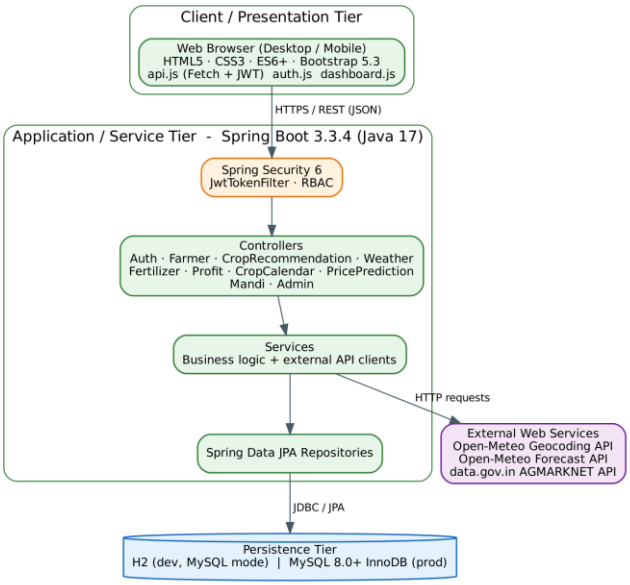


Figure 2: Layered system architecture of Smart Krishi

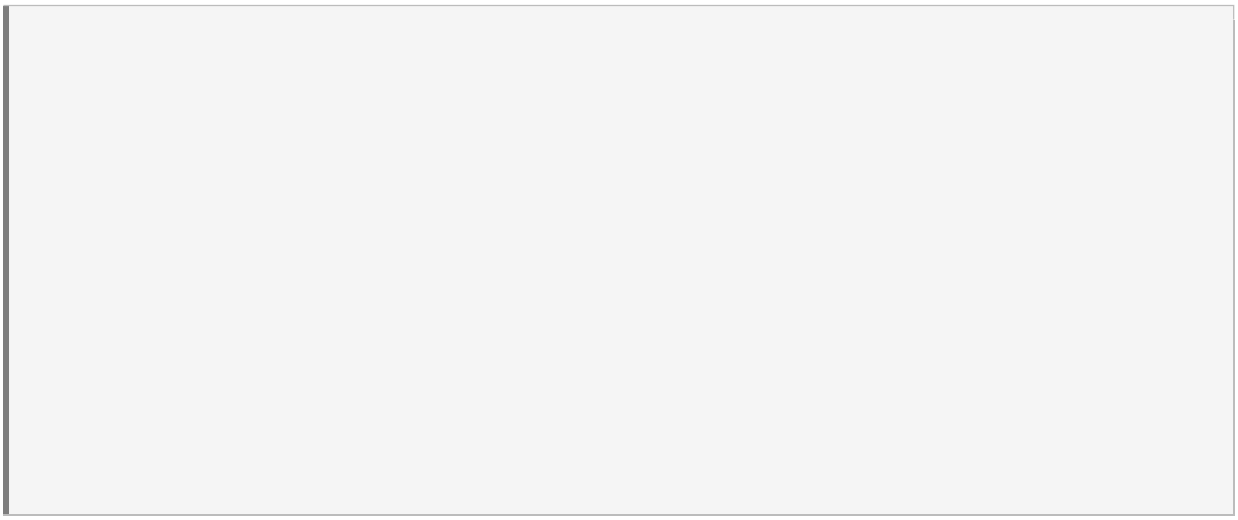
4.2 Layer Responsibilities

Table 7: Responsibilities of each architectural layer

Layer	Responsibility
Presentation	Static HTML5 pages styled with CSS3 and Bootstrap 5.3; ES6+ scripts handle events, validation, toasts (UI.showToast) and rendering. api.js centralises Fetch calls and injects the JWT; auth.js handles login and registration; dashboard.js renders multi-plot data.
Security	SecurityConfig and JwtTokenFilter validate Bearer tokens on every request, set the security context and enforce role rules without server-side sessions.
Controller	REST controllers (Auth, Farmer, CropRecommendation, Weather, Fertilizer, ProfitCalculator, CropCalendar, PricePrediction, Mandi, Admin) validate input DTOs and delegate to services.
Service	Business rules and algorithms; clients for Open-Meteo and AGMARKNET; MandiLocationRegistry for registered market coordinates.
Repository / Persistence	Spring Data JPA repositories over MySQL 8.0 InnoDB in production and H2 in MySQL mode in development; six tables with referential integrity.

4.3 Project Structure

The repository is split into a static frontend and a Spring Boot backend, as recorded in the project documentation:



4.4 Module Interaction

Although each module is independently usable, the data flows naturally from one to the next: the farmer profile supplies location and soil context; the crop recommendation selects a crop that feeds the fertilizer advisor and the crop calendar; fertilizer cost feeds the profit calculator; expected price links to the price prediction and to the mandi locator. The administrator seeds the market price data that underpins prediction and the verified fallback.

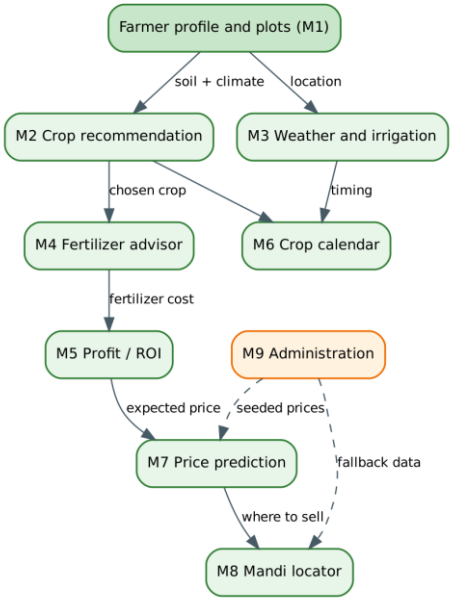


Figure 3: Data flow between the nine modules

4.5 Database Design

The relational schema consists of six core tables. Referential integrity is enforced by foreign keys: deleting a user cascades to the farmer profile and, through it, to all farm plots; deleting a user sets user_id to NULL in crop_recommendations so that historical recommendations are retained. The market_prices and advisories tables are standalone reference data maintained by the administrator.

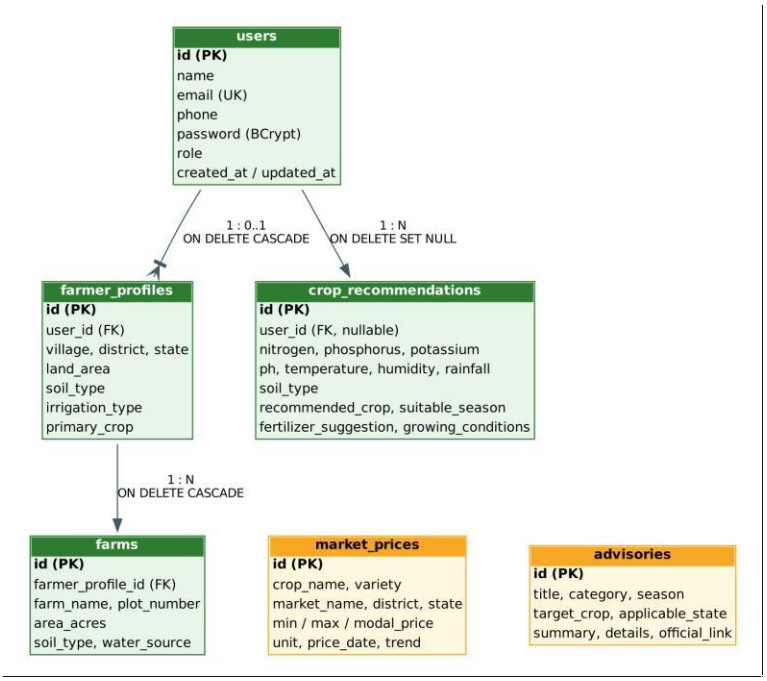


Figure 4: Entity-relationship diagram (six core tables)

Data dictionary

Table 8: Table: users

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Unique user key
name	VARCHAR(100)	NO	Full name
email	VARCHAR(120)	NO (UK)	Login identifier
phone	VARCHAR(20)	NO	Mobile number
password	VARCHAR(255)	NO	BCrypt hash
role	VARCHAR(20)	NO	ROLE_FARMER or ROLE_ADMIN
created_at, updated_at	TIMESTAMP	YES	Audit timestamps

Table 9: Table: farmer_profiles

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Profile key
user_id	BIGINT	NO (FK)	users.id, ON DELETE CASCADE
village, district, state	VARCHAR(100)	YES	Location
land_area	DOUBLE	YES	Total holding in acres

Column	Type	Nullable	Description
soil_type	VARCHAR(50)	YES	Predominant soil
irrigation_type	VARCHAR(50)	YES	Canal, tube well, drip, rainfed
primary_crop	VARCHAR(100)	YES	Main crop

Table 10: Table: farms

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Plot key
farmer_profile_id	BIGINT	NO (FK)	farmer_profiles.id, ON DELETE CASCADE
farm_name	VARCHAR(100)	NO	Plot label
plot_number	VARCHAR(50)	YES	Survey / khasra number
area_acres	DOUBLE	NO	Plot area
soil_type	VARCHAR(50)	YES	Plot soil
water_source	VARCHAR(100)	YES	Dedicated water source

Table 11: Table: crop_recommendations

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Record key
user_id	BIGINT	YES (FK)	users.id, ON DELETE SET NULL
nitrogen, phosphorus, potassium	DOUBLE	NO	Soil nutrients in kg/ha
ph, temperature, humidity, rainfall	DOUBLE	NO	Soil and climate inputs
soil_type	VARCHAR(50)	NO	Soil tested
recommended_crop	VARCHAR(100)	NO	Prescribed crop
suitable_season	VARCHAR(50)	YES	Kharif, Rabi, Zaid
fertilizer_suggestion, growing_conditions	TEXT	YES	Initial advisory and notes

Table 12: Table: market_prices

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Record key
crop_name, variety	VARCHAR(100)	NO / YES	Commodity and grade
market_name, district, state	VARCHAR(100-150)	NO	APMC market yard
min_price, max_price, modal_price	DOUBLE	NO	Prices in rupees
unit	VARCHAR(30)	YES	e.g. Rs/Quintal
price_date	DATE	NO	Trading date
trend	VARCHAR(20)	YES	RISING, FALLING, STABLE

Table 13: Table: advisories

Column	Type	Nullable	Description
id	BIGINT AUTO_INCREMENT	NO (PK)	Record key
title	VARCHAR(255)	NO	Scheme or advisory title
category	VARCHAR(50)	NO	SCHEME, FERTILIZER, PEST_CONTROL ...
season, target_crop	VARCHAR	YES	Season and crop targeted
summary, details	TEXT	YES / NO	Guideline text
applicable_state, official_link	VARCHAR	YES	Jurisdiction and portal link

4.6 REST API Design

Table 14: REST endpoints defined in the SRS

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a farmer; password stored as BCrypt hash
POST	/api/auth/login	Public	Verify credentials and issue JWT
GET / PUT	/api/farmers/{id}	Farmer	Read or update farmer profile
POST	/api/farmers/{id}/farms	Farmer	Add a farm plot
DELETE	/api/farmers/farms/{farmId}	Farmer	Remove a farm plot
GET	/api/farmers/{id}/recommendations	Farmer	Recommendation history
POST	/api/crops/recommend	Public	Crop recommendation
GET	/api/weather?city= or ?latitude=&longitude=	Public	Live weather and irrigation advisory
POST	/api/fertilizer/recommend	Public	Nutrient deficits, bags, amendments, cost
POST	/api/profit/calculate	Public	Revenue, net profit, ROI, break-even
GET	/api/crop-calendar/{crop}	Public	Six-stage calendar (optional ?state=)
GET	/api/price-prediction/{crop}	Public	Trend and forecast
GET	/api/mandi/nearby, /api/mandi/status	Public	Nearby mandis; service health
POST	/api/market/prices, /api/advisory	Admin	Publish prices and schemes
GET / DELETE	/api/admin/stats, /api/admin/farmers, /api/admin/farmers/{userId}	Admin	Statistics and farmer audit

4.7 Authentication Flow

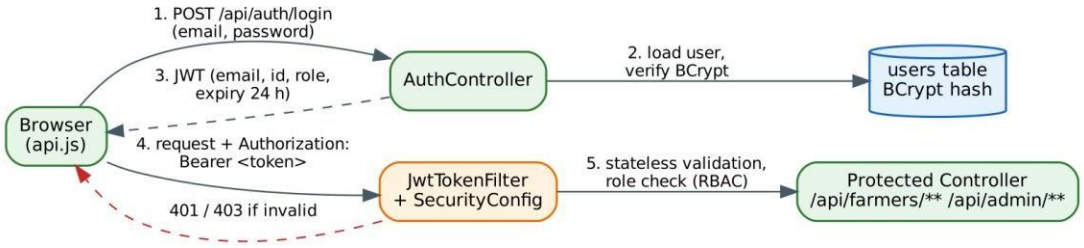


Figure 5: Stateless JWT authentication and authorisation flow

4.8 User Interface Design

The interface follows an Agro Forest Green identity: primary colour #2e7d32, secondary accent #4caf50, background #f4fbf4 and neutral borders #e0e0e0, with Inter typography and Bootstrap Icons 1.11.3. The layout uses Bootstrap 5.3 grid classes (col-12, col-md-6, col-lg-4) to adapt from 320 px mobile to 1920 px desktop widths. Interactive feedback includes real-time validation, toast notifications and loading spinners during asynchronous calls.

4.9 Deployment Architecture

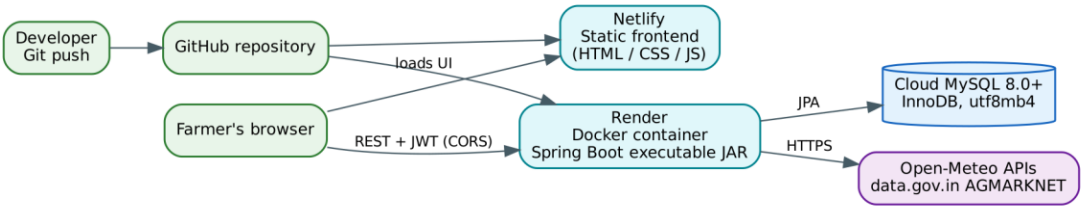


Figure 6: Cloud deployment architecture

5 Implemention Of Modules

This chapter describes the implementation of each module: its purpose, inputs, algorithm, endpoints, the challenges faced and their resolution, and the corresponding screenshots of the running application and of the source code. Class and file names follow the traceability matrix in the SRS.

5.1 Module 1: Farmer Management and Land Profiling

Developed by Ravina (Team Leader and Tech Lead). The module manages identity, JWT issuance, BCrypt password hashing and multi-plot agricultural profiles. Backend components: AuthController, FarmerController and FarmerService; frontend pages: login.html, register.html and profile.html with auth.js and dashboard.js.

Registration accepts name, email, phone and password and stores a BCrypt hash with cost factor 10. Login verifies the credential and issues a stateless JWT containing the user's email, ID, role and an expiry (default 24 hours). A farmer maintains a profile (village, district, state, land area, soil type, irrigation type, primary crop) and any number of farm plots, each with a name, plot number, area in acres, soil type and water source.

Challenges and resolution. Keeping the JWT completely stateless required a custom Spring Security filter chain. Cascading deletes were configured so that removing a user automatically clears the farmer profile and associated plots without violating foreign-key constraints.

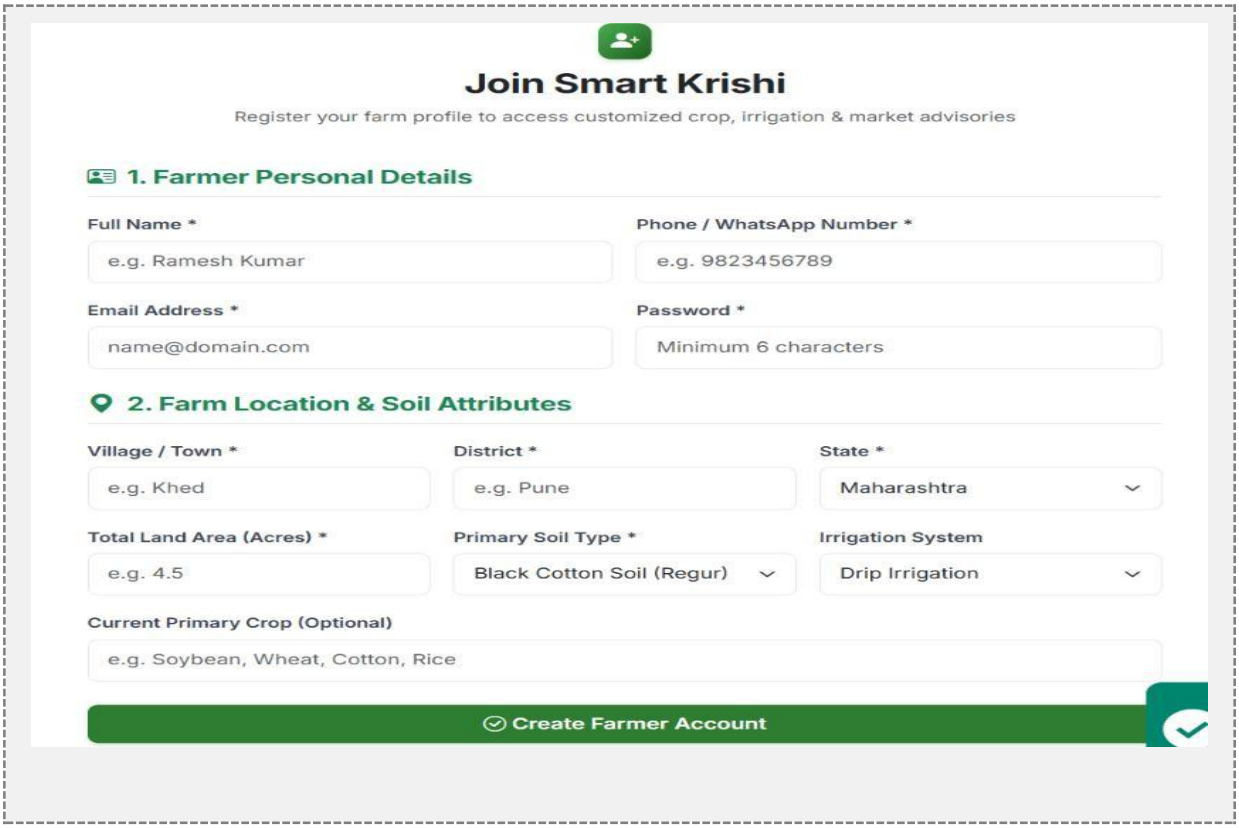


Figure 7: Registration and login pages

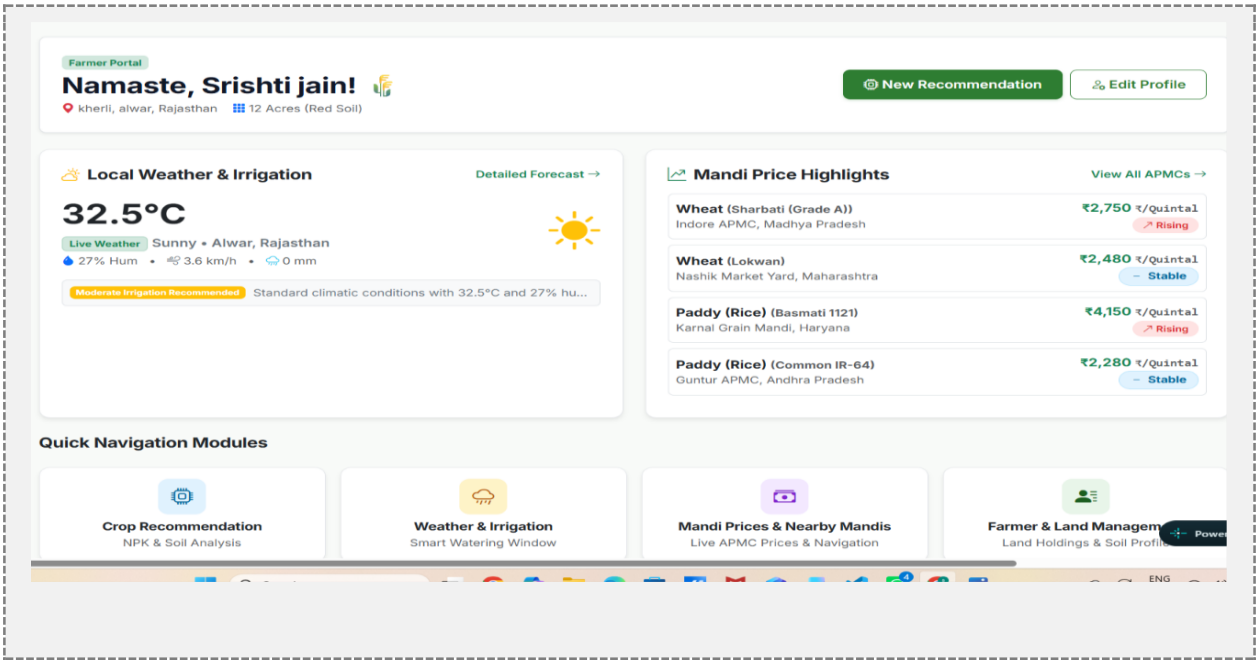


Figure 8: Farmer dashboard with multi-plot cards

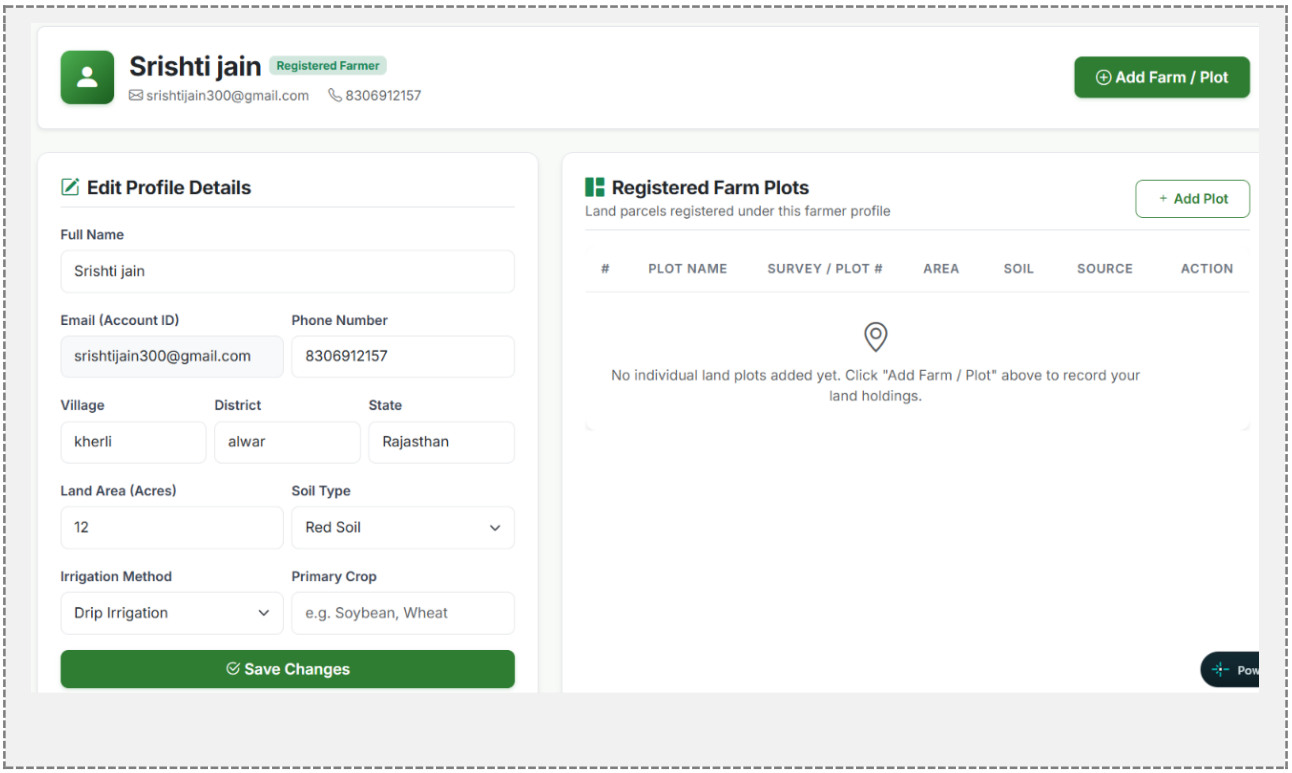


Figure 9: Farmer And Land Management

5.2 Module 2: Precision Crop Recommendation Engine

Developed by Yashvardhan Singhal. The engine prescribes agronomically suitable crops from chemical soil composition, climate and soil texture. Backend: CropRecommendationController and CropRecommendationService; frontend: crop-recommendation.html and crop-recommendation.js.

Table 15: Input validation for crop recommendation (FR-2.1)

Input	Valid range	Notes
Nitrogen (N)	0.0 to 300.0 kg/ha	Soil available nitrogen
Phosphorus (P)	0.0 to 200.0 kg/ha	
Potassium (K)	0.0 to 300.0 kg/ha	
pH	3.0 to 11.0	Soil reaction
Temperature	-10.0 to 60.0 C	
Humidity	0.0 to 100.0 %	
Rainfall	0.0 to 3000.0 mm	
Soil type	Alluvial, Black, Red, Clay, Sandy, Laterite	Categorical

The engine evaluates agronomic suitability matrices across ten crops: rice, wheat, cotton, maize, sugarcane, chickpea, mustard, groundnut, soybean and tomato. For example, high rainfall (above 200 mm) combined with clay soil and high nitrogen prescribes rice. The response contains the recommended crop, suitable season (Kharif, Rabi or Zaid), growing conditions, a fertilizer suggestion

and crop details. Authenticated recommendations are persisted in crop_recommendations and exposed through the farmer's history endpoint. Because the logic is explicit, every recommendation can be traced to its rule, unlike the learned models in the literature [1]-[4].

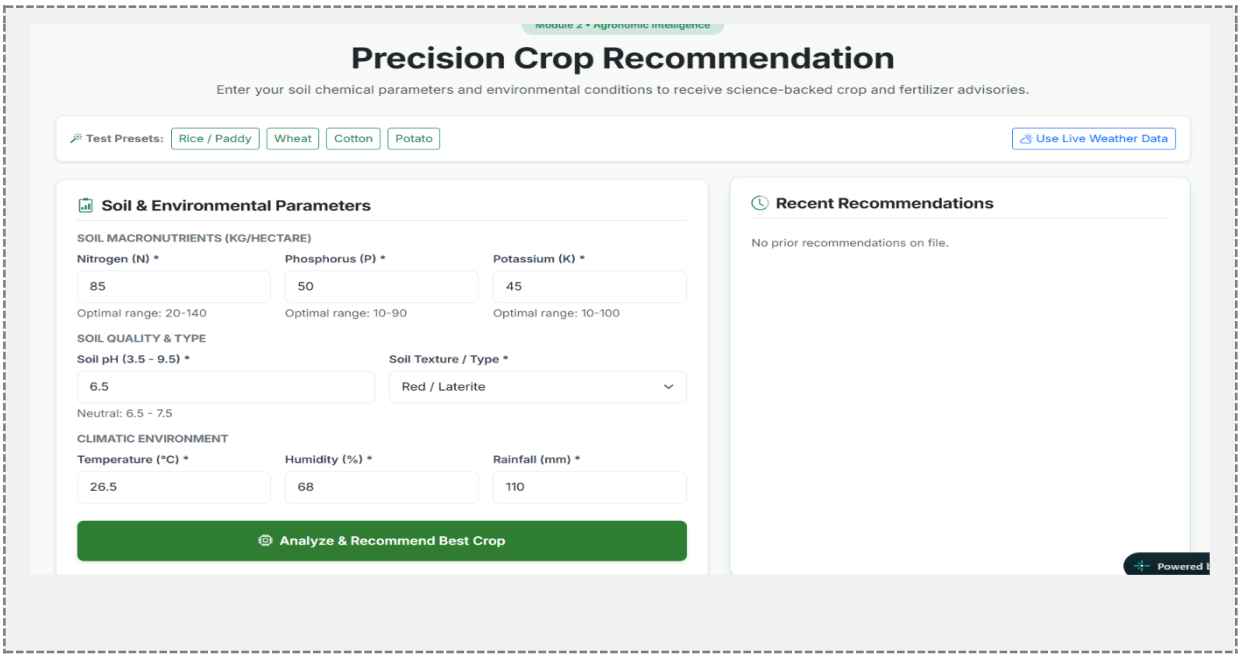


Figure 10: Crop recommendation form and result

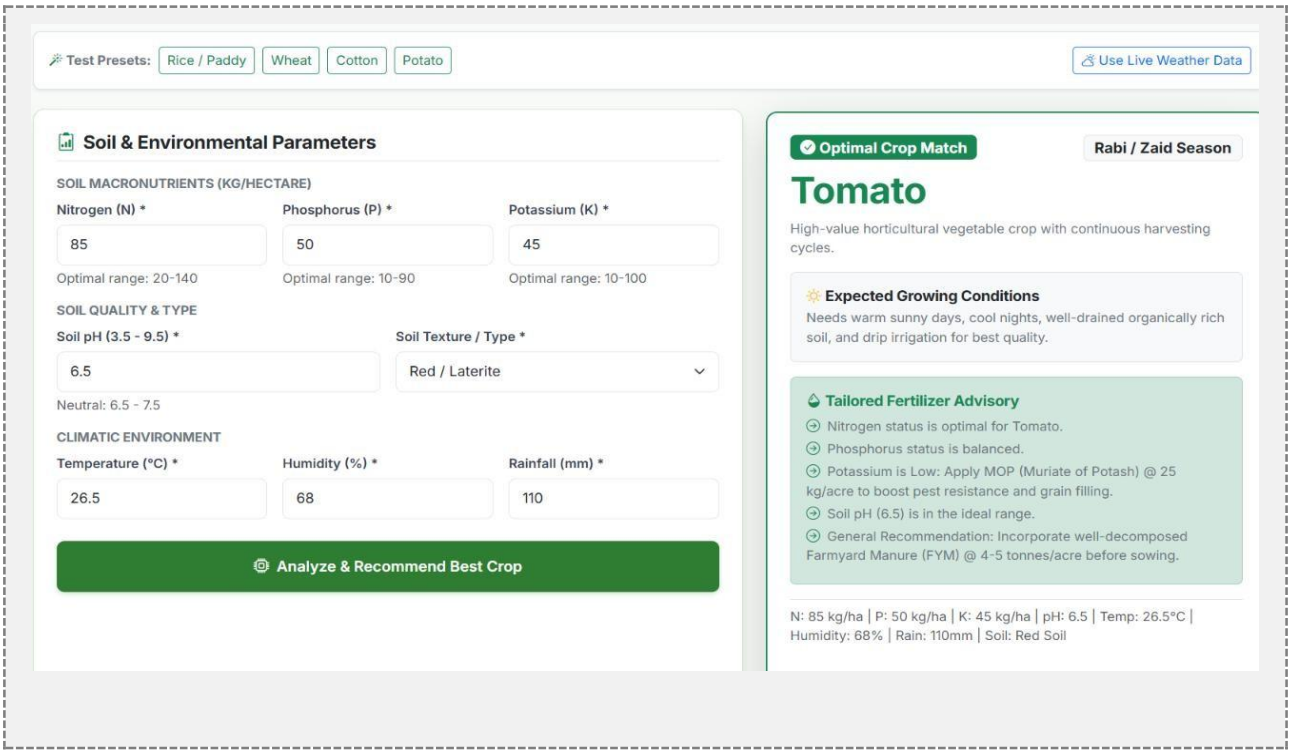


Figure 11: CropRecommendationService: suitability matching

5.3 Module 3: Live Weather and Smart Irrigation Advisory

Developed by Yashvardhan Singhal. The module delivers real-time, location-based weather and irrigation guidance without simulated values. Backend: WeatherController and WeatherService; frontend: weather.html and dashboard.html.

Location is resolved either from browser GPS (latitude and longitude) or from a city name, which requires sequential chaining: the Open-Meteo Geocoding API first converts the name to coordinates, and then the Forecast API is called. An unknown city raises ResourceNotFoundException (HTTP 404) with an explicit message. The response includes temperature, condition, humidity, wind speed, rainfall, an observation timestamp and a five-day forecast of daily minimum and maximum temperatures, rainfall probability and WMO condition codes.

Table 16: Smart irrigation decision model (FR-3.4)

Condition	Irrigation advice
Rainfall > 5 mm	PAUSE IRRIGATION (prevent root rot and waterlogging)
Temperature > 35 C and humidity < 40%	HEAVY IRRIGATION REQUIRED (early morning or evening)
Humidity > 85%	LIGHT / SKIP IRRIGATION
Otherwise	MODERATE IRRIGATION RECOMMENDED

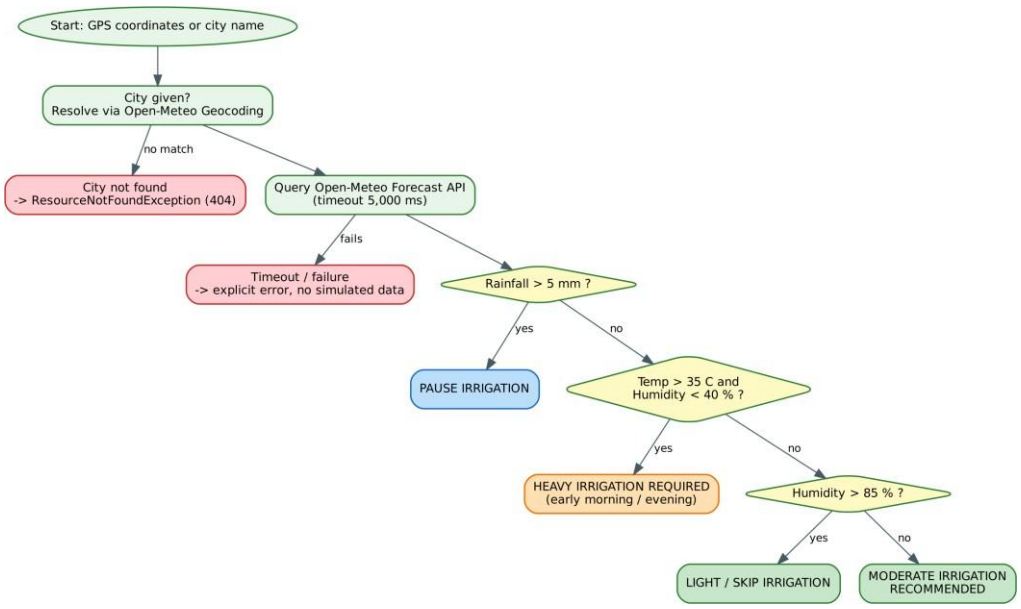


Figure 12: Weather retrieval and irrigation decision flow

Challenges and resolution. The strict zero-fake-data requirement meant API failures had to be handled gracefully: queries time out after 5,000 ms and return an appropriate not-found error rather than substituted values. This directly addresses the credibility concern raised for forecast advisories in [8].

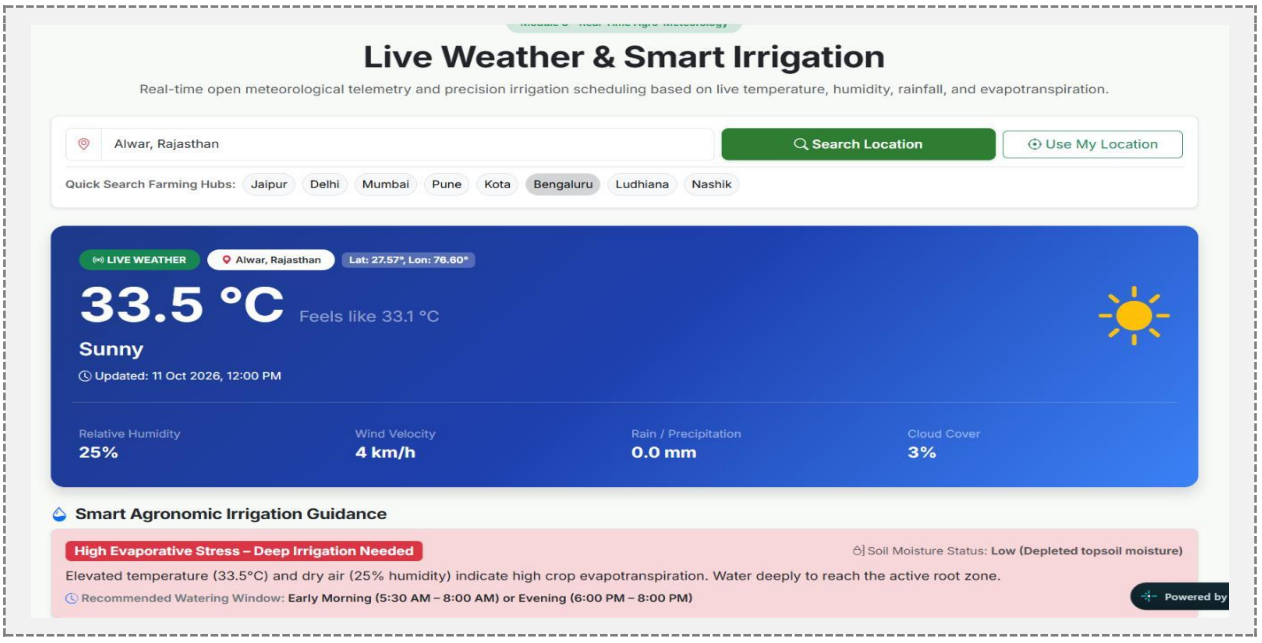


Figure 13: Weather and irrigation advisory page

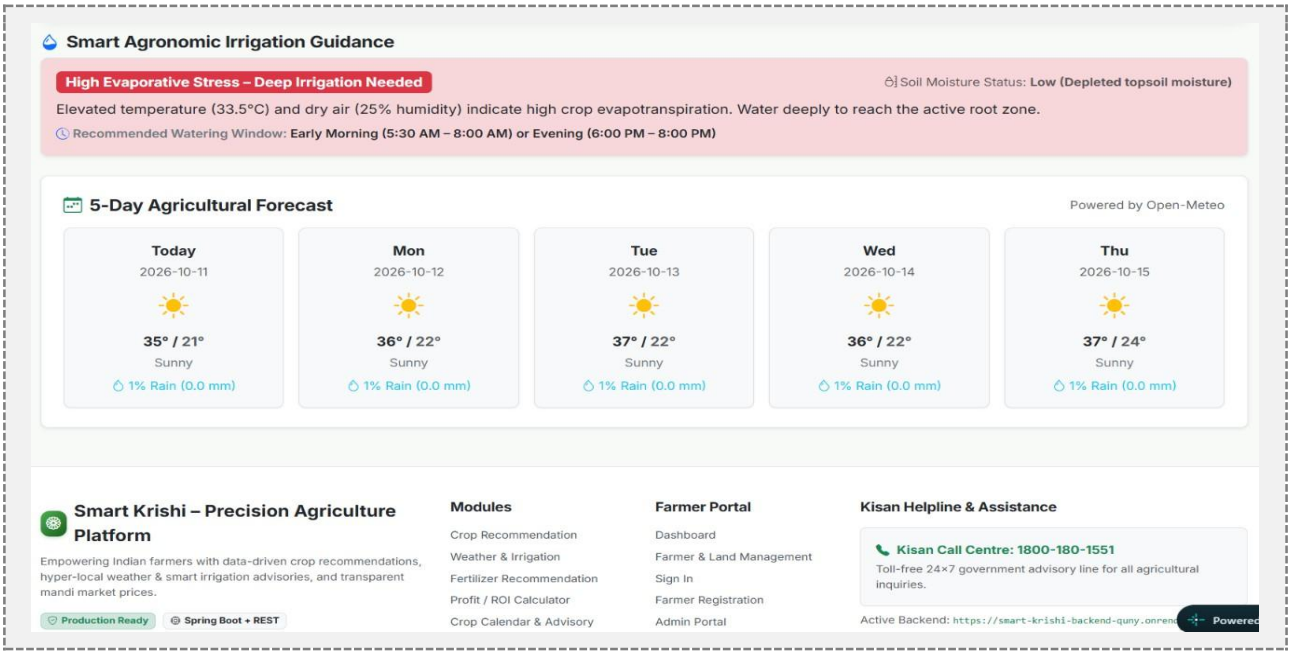


Figure 14: WeatherService: geocoding, forecast call and irrigation rules

5.4 Module 4: Smart Fertilizer Recommendation Advisor

Developed by Team Member 3. The advisor calculates fertilizer quantities aligned with ICAR practice and advises on pH correction. Backend: FertilizerController and FertilizerService; frontend: fertilizer.html and fertilizer.js.

Elemental deficits are computed for each nutrient as the positive difference between the crop's target and the soil value:

DeltaN = max(0, N_target - N_soil); DeltaP = max(0, P_target - P_soil); DeltaK = max(0, K_target - K_soil)

Table 17: Fertilizer grades, bag sizes and subsidised MRP used for estimation

Product	Grade	Bag size	MRP (Rs/bag)	Nutrient
Urea	46% N	50 kg	270	Nitrogen
DAP	18% N, 46% P2O5	50 kg	1,350	Phosphorus
MOP	60% K2O	50 kg	1,700	Potassium
SSP	16% P2O5	50 kg	450	Phosphorus
Zinc sulphate	21% Zn	25 kg	750	Micronutrient

Required nutrients are converted into commercial bag counts, a basal application percentage at sowing and split top-dressing percentages at the vegetative and flowering stages are returned, and an approximate total cost is obtained by multiplying bag counts by the MRP. Soil amendments follow strict boundaries: pH below 6.0 prescribes agricultural lime (CaCO3); pH above 8.0 prescribes gypsum (CaSO4.2H2O) and green manuring.

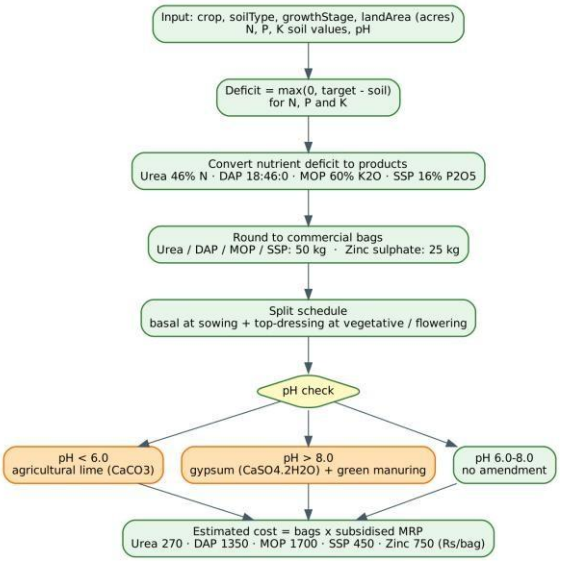


Figure 15: Fertilizer calculation and soil amendment flow

Challenges and resolution. Translating raw chemical deficits into standard Indian bag sizes required careful rounding logic, and categorising soil amendments safely required strict boundary validation. Tests such as testFertilizerAcidicAndAlkalineSoilWarnings confirm that the backend never recommends a dangerous chemical load. Expressing advice as physical bags targets the actionability gap identified in [14].

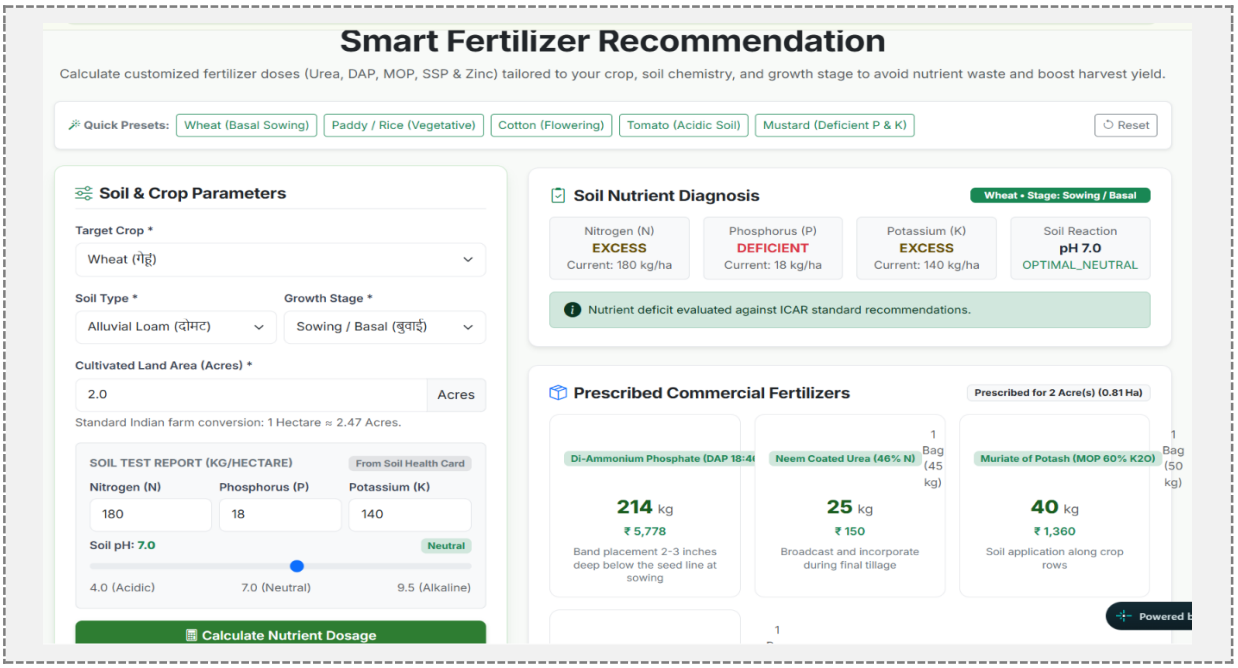


Figure 16: Fertilizer advisor input and result

Figure 17: FertilizerService: deficit to bag conversion and pH amendments

5.5 Module 5: Crop Profit and ROI Calculator

Developed by Team Member 3. The calculator provides harvest financial projections, cost structure and return on investment. Backend: ProfitCalculatorController and ProfitCalculatorService; frontend: profit-calculator.html and profit-calculator.js.

Inputs are the crop name, land area in acres, expected yield per acre in quintals, expected selling price per quintal and seven cost heads: seed, fertilizer, pesticide, labour, irrigation, machinery and other. The calculations follow the cost-concept approach in [17]:

$$\text{Total Yield} = \text{Land Area} \times \text{Yield per Acre}$$

$$\text{Gross Revenue} = \text{Total Yield} \times \text{Selling Price per Quintal}$$

$$\text{Total Cost} = \text{Seed} + \text{Fertilizer} + \text{Pesticide} + \text{Labour} + \text{Irrigation} + \text{Machinery} + \text{Other}$$

$$\text{Net Profit} = \text{Gross Revenue} - \text{Total Cost}; \quad \text{Profit per Acre} = \text{Net Profit} / \text{Land Area}$$

$$\text{ROI (\%)} = (\text{Net Profit} / \text{Total Cost}) \times 100; \quad \text{Break-even Price per Quintal} = \text{Total Cost} / \text{Total Yield}$$

Table 18: Financial health classification (FR-5.6)

ROI range	Class	Interpretation
Above 100%	EXCELLENT	Very strong return
50% to 100%	GOOD	Healthy margin
20% to 50%	MODERATE	Acceptable, sensitive to price
0% to 20%	LOW MARGIN	Thin margin
Below 0%	LOSS MAKING	Costs exceed revenue

Worked example (illustrative values)

Land 2 acres; expected yield 20 quintals per acre; selling price Rs 2,200 per quintal; costs: seed 6,000, fertilizer 12,000, pesticide 5,000, labour 18,000, irrigation 7,000, machinery 9,000 and other 3,000. Total yield = 40 quintals; gross revenue = Rs 88,000; total cost = Rs 60,000; net profit = Rs 28,000 (Rs 14,000 per acre); ROI = 46.7%, class MODERATE; break-even price = Rs 1,500 per quintal. These figures are only a demonstration of the formulas and are not market data.

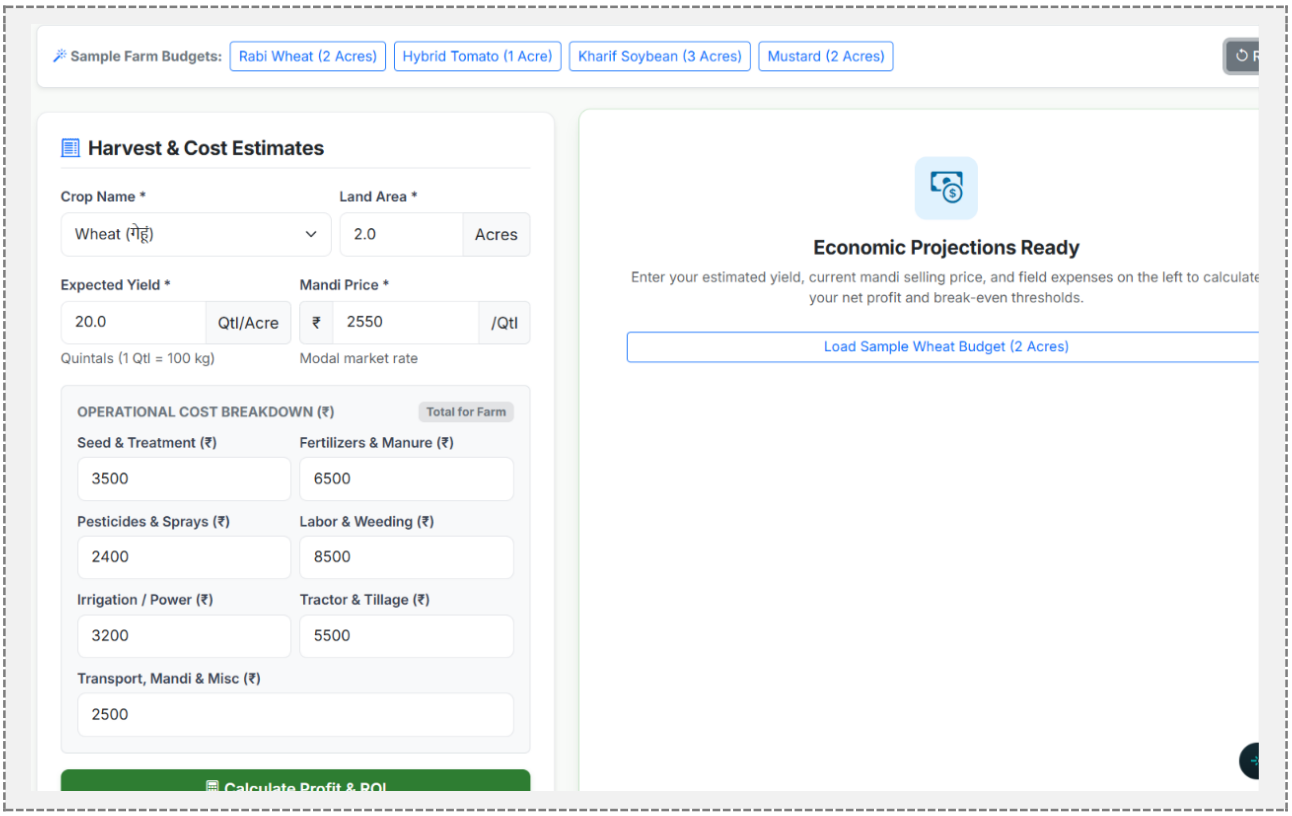


Figure 18: Profit and ROI calculator

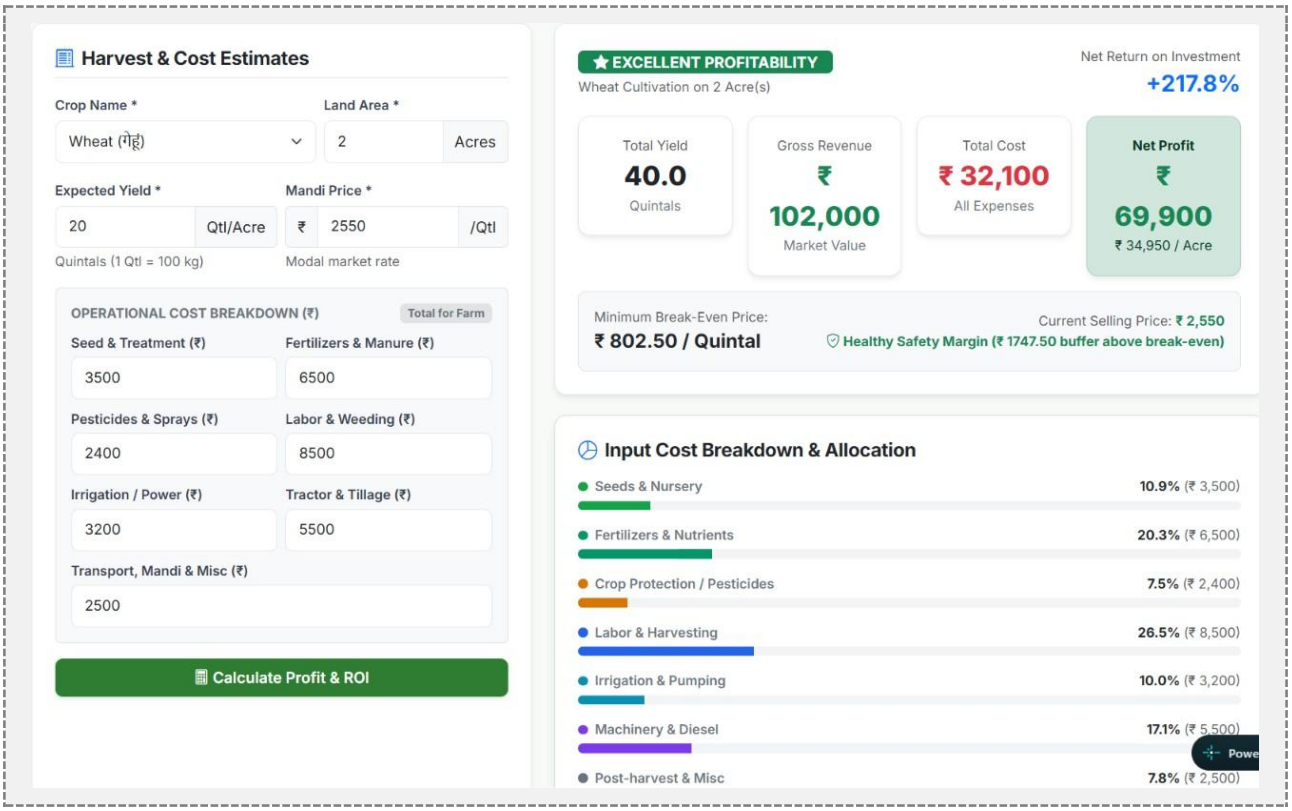


Figure 19: ProfitCalculatorService: revenue, ROI and break-even

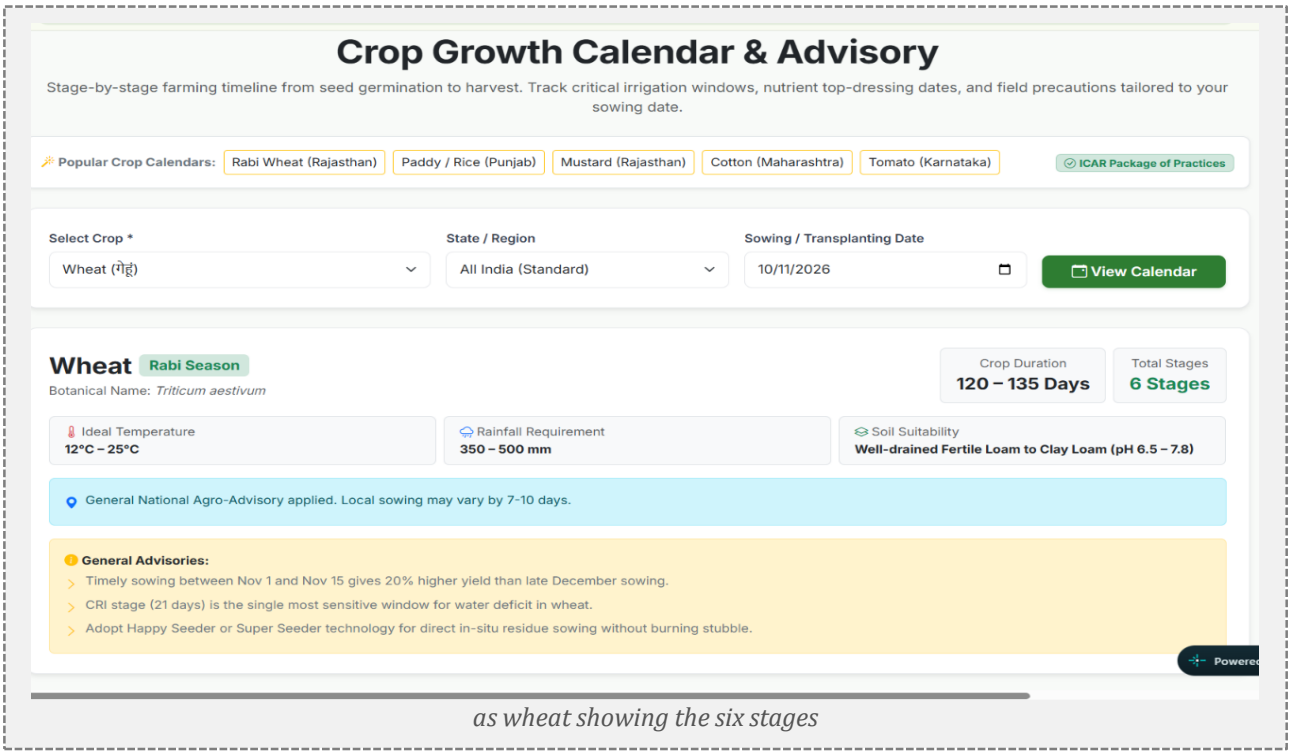
5.6 Module 6: Crop Growth Calendar and Phenological Advisory

Developed by Team Members 4 and 5. The module generates a structured timeline of crop development with critical action items. Backend: CropCalendarController and CropCalendarService; frontend: crop-calendar.html and crop-calendar.js.

Table 19: Six-stage crop calendar (FR-6.1)

#	Phenological stage	Typical content
1	Sowing and basal nutrition	Seed bed, sowing window, basal fertilizer
2	Germination and seedling establishment	Early irrigation, gap filling
3	Vegetative growth, tillering / branching	Top-dressing, weeding
4	Flowering, tassel / pod emergence	Water-sensitive stage, protection
5	Grain filling and maturation	Irrigation scheduling, pest watch
6	Harvesting and storage	Harvest timing, drying, storage

For each stage the response gives day ranges, irrigation needs, fertilizer top-dressing milestones and protective field precautions. A state query parameter (for example ?state=Rajasthan) appends a tailored ICAR state package of practices. If an unrecognised crop is requested, the system returns a balanced standard 120-day timeline instead of an error.



as wheat showing the six stages

Figure 20: Crop growth calendar

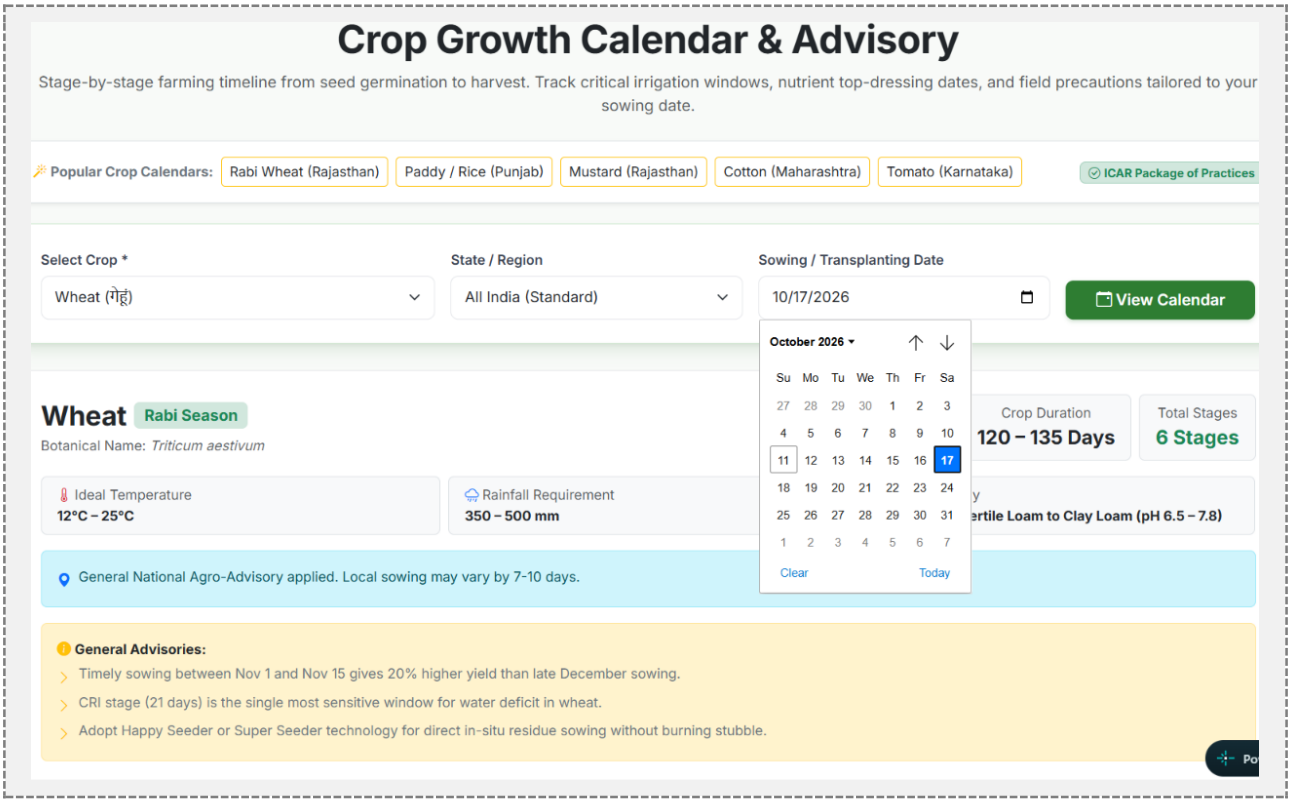


Figure 21: CropCalendarService: stage generation and state advisory

5.7 Module 7: Crop Price Trend Prediction Engine

Developed by Team Members 4 and 5. The engine computes short-to-medium-term price projections from historical APMC time-series data stored in market_prices. Backend: PricePredictionController and PricePredictionService; frontend: price-prediction.html and price-prediction.js.

The service retrieves the crop's historical records over the requested horizon (default 10 days) and requires at least three data points; otherwise it returns status INSUFFICIENT_DATA with a user-friendly explanation rather than an error. It then computes:

$$SMA = (1/N) \times Sum(P_i);\ \ Momentum\ dP = (P_{latest} - P_{earliest}) / dt;\ \ P_{pred} = P_{latest} + (dP \times horizonDays \times 0.5)$$

The trend is classified as RISING, FALLING or STABLE, a confidence score between 50% and 95% is computed in proportion to sample size and consistency, and every response carries a statutory agricultural market-risk disclaimer. As an illustration with invented numbers: if the earliest price in a 10-day window is Rs 2,000 and the latest Rs 2,200, momentum is Rs 20 per day and a 10-day forecast is 2,200 + 20 x 10 x 0.5 = Rs 2,300 per quintal.

This method is deliberately simple and transparent. The literature uses ARIMA, ARFIMAX and machine-learning models [18]-[21], which are more rigorous; upgrading to ARIMA is the first item of future work (Chapter 11).

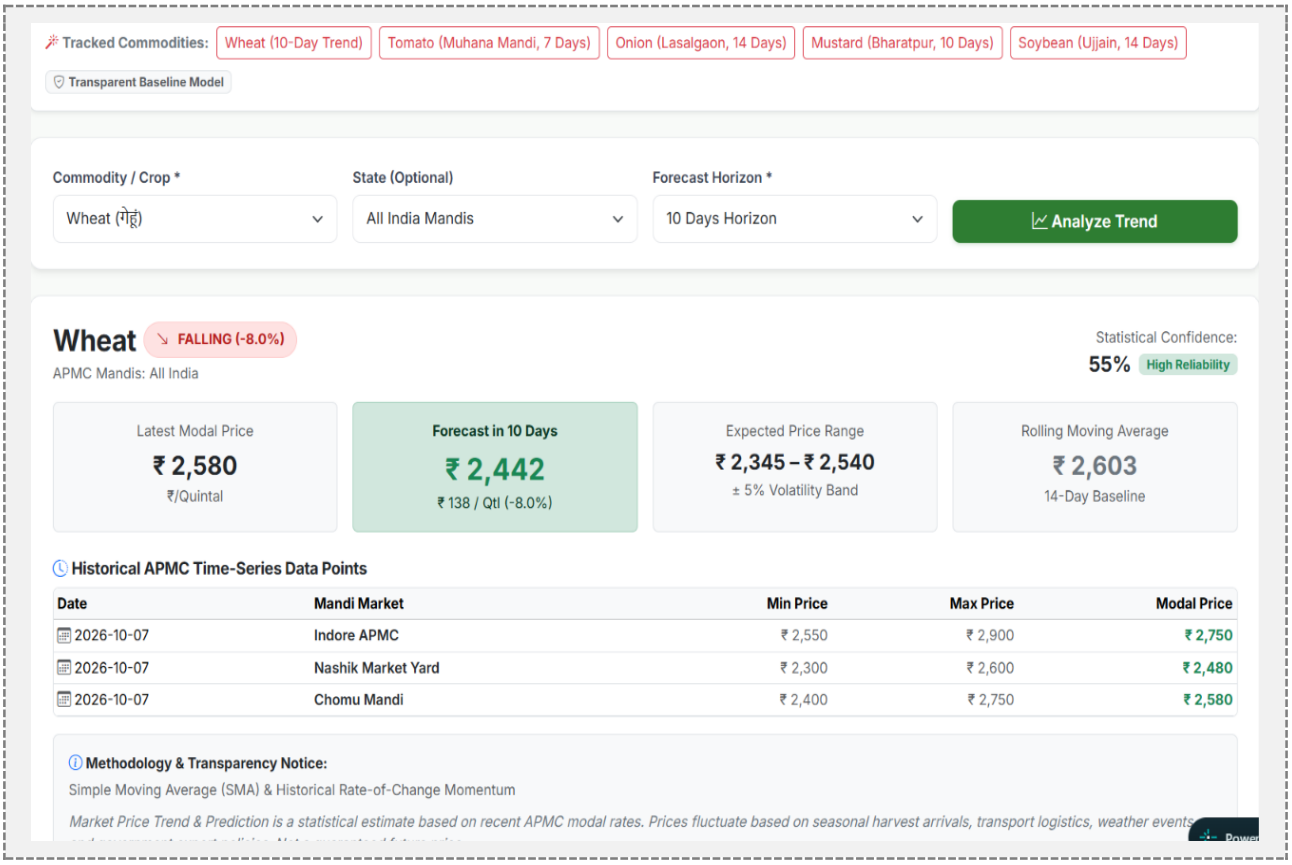


Figure 22: Price prediction result

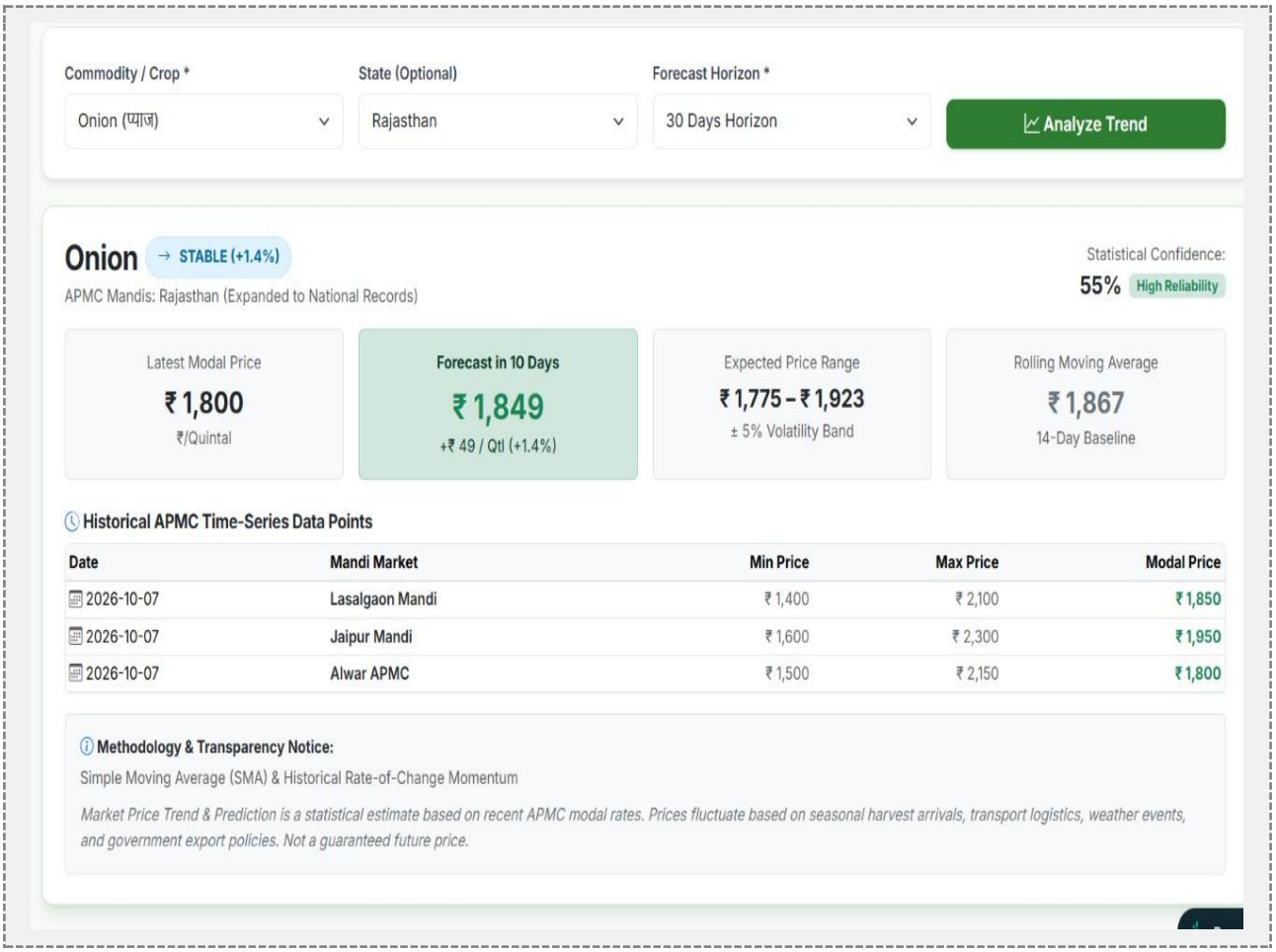


Figure 23: PricePredictionService: moving average, momentum and forecast

5.8 Module 8: Nearby APMC Mandi Prices and Geodesic Locator

Developed by Team Members 4 and 5. The module finds the closest operational APMC market yards and shows transparent prices. Backend: MandiController, MandiService and MandiLocationRegistry; frontend: market-prices.html and market-prices.js.

The great-circle distance between the user and each registered mandi is computed with the Haversine formula:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right), \quad R = 6371 \text{ km}$$

Farmers choose a radius of 10, 25, 50 (default) or 100 km and mandis beyond it are strictly excluded. If GPS is unavailable the farmer may enter a state and district manually. Results can be sorted by nearest distance, highest modal price or lowest modal price, and each includes a Google Maps navigation link of the form https://www.google.com/maps/dir/?api=1&destination={lat},{lon}. A status endpoint (GET /api/mandi/status) reports government API connectivity and fallback counts.

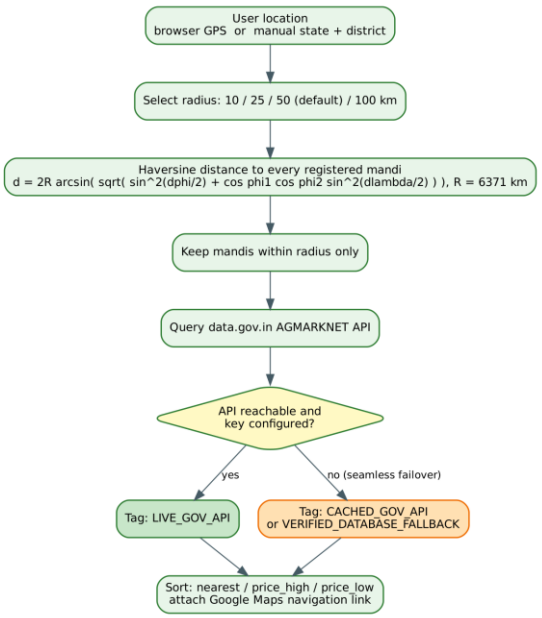


Figure 24: Mandi search with live-data and fallback paths

Challenges and resolution. The AGMARKNET API frequently experienced downtime or lacked a configured key. The service therefore switches seamlessly from LIVE_GOV_API to cached data or the VERIFIED_DATABASE_FALLBACK without interrupting the user, and every response declares its origin tag (LIVE_GOV_API, CACHED_GOV_API or VERIFIED_DATABASE_FALLBACK). The test testNearbyMandiPricesJaipur verifies that the interface displays the cached tag when the live connection fails. This is an application of the transparent-sourcing constraint; given the mixed e-NAM evidence [24], [25], the locator is positioned as an information tool rather than a guarantee of better prices.

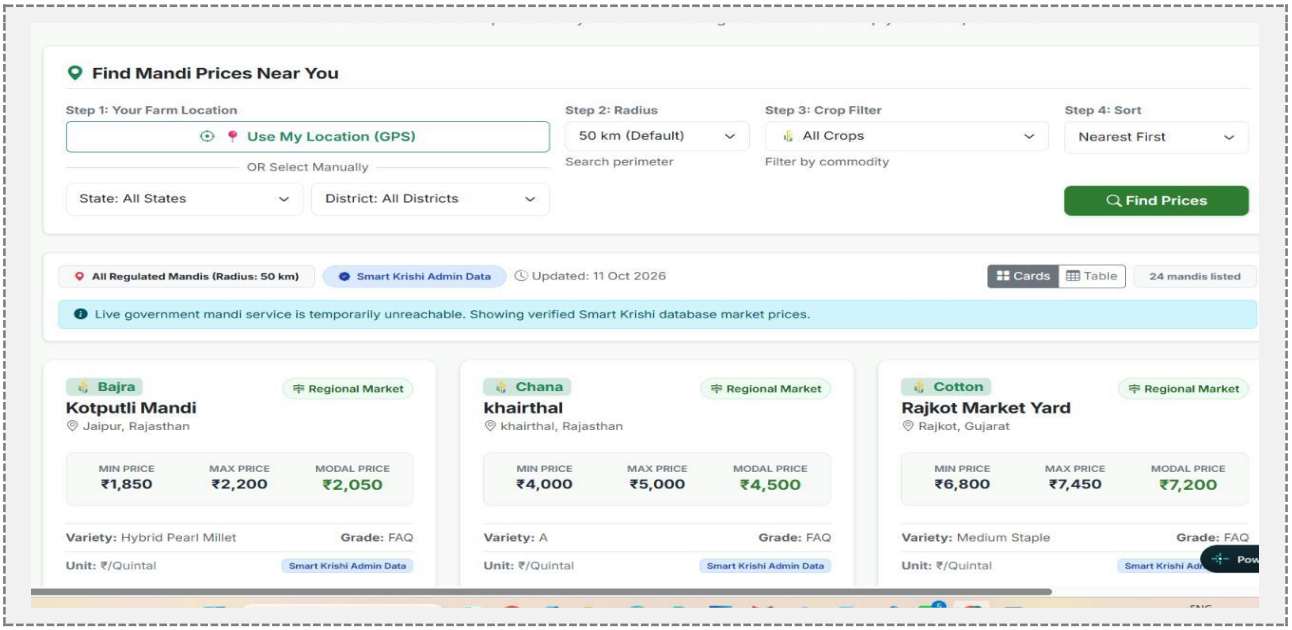


Figure 25: Nearby mandi search results with data-source tag

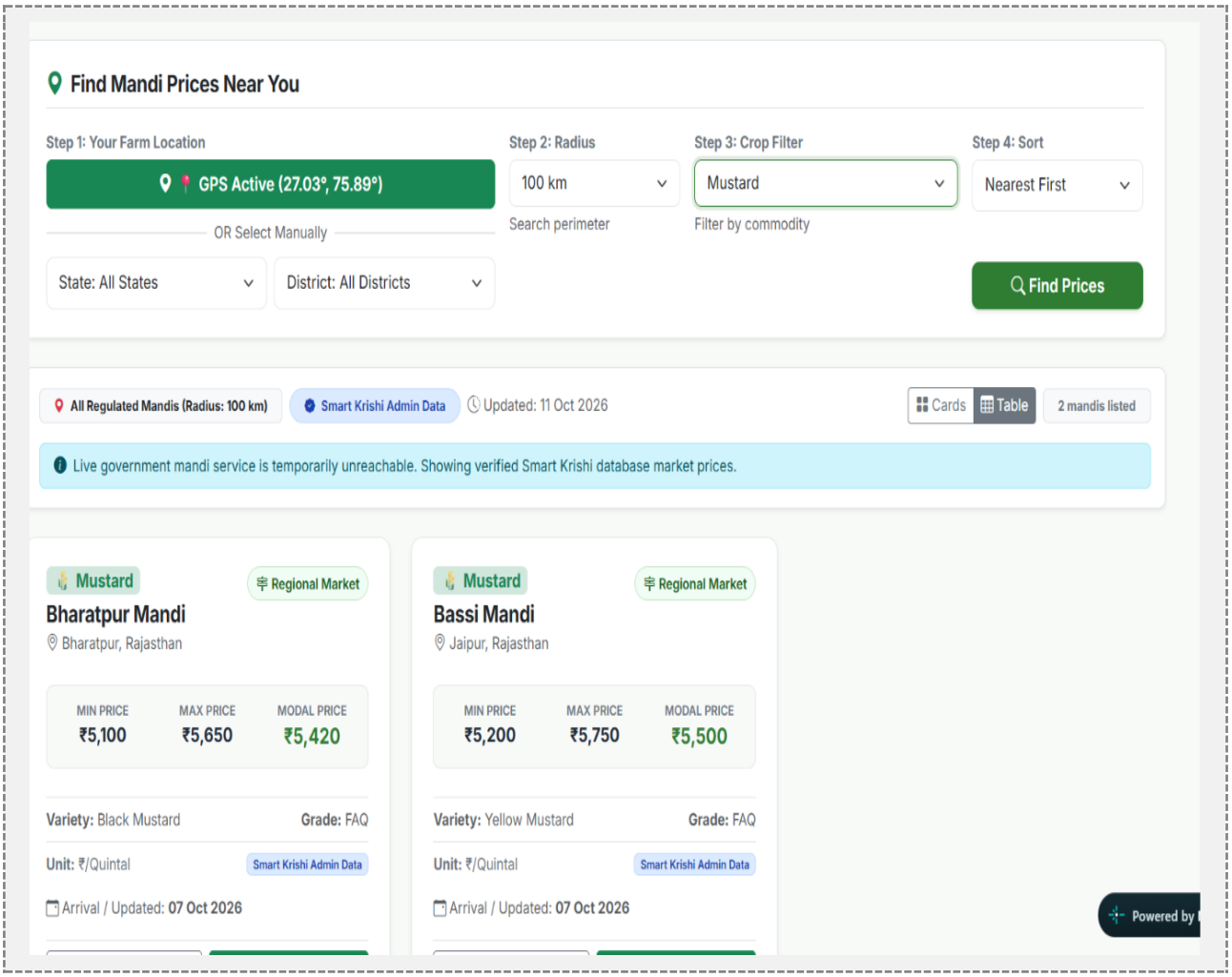


Figure 26: MandiService: Haversine distance and fallback logic

5.9 Module 9: System Administration and Governance

The administrator portal provides centralised governance. Backend: AdminController and AdminService; frontend: admin.html and admin.js. Features: aggregate statistics (registered farmers, total recommendations, market price records and advisories) at /api/admin/stats; a farmer directory with account removal (administrators cannot delete themselves); creation of new APMC price records with crop, variety, mandi, district, state, minimum, maximum and modal price and trend; and publication of government schemes such as PM-KISAN, PMFBY and the Soil Health Card with category, season, target crop, details and official URL. Access requires ROLE_ADMIN, and unauthenticated requests to /api/admin/ routes are rejected, as verified in the integration tests.

5.10 Responsive Design

The interface was built with Bootstrap 5.3 without a heavy frontend framework so that pages load quickly on 4G connections. The same pages adapt from smartphone to desktop widths.

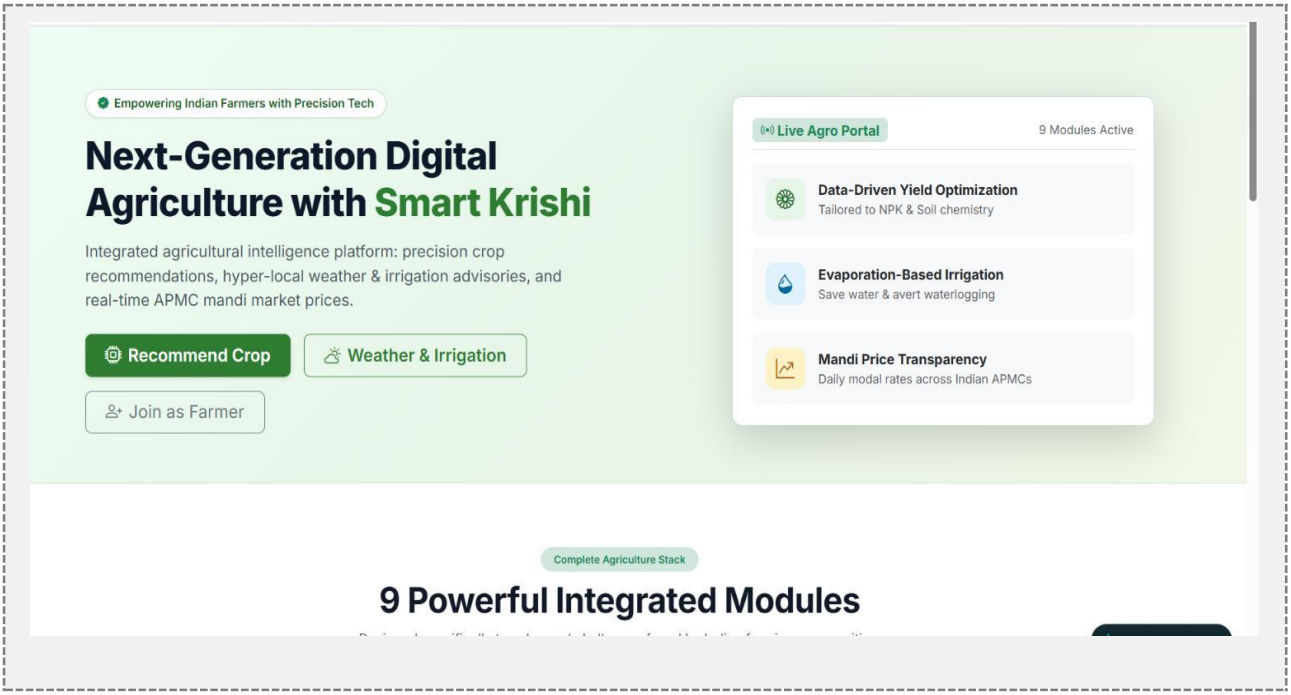


Figure 29: Mobile view of the application

6 Security Design

6.1 Authentication and Token Management

Smart Krishi enforces stateless JWT role-based access control through Spring Security 6. On login the server verifies the BCrypt hash and issues a signed JWT carrying the user's email, ID, role and expiry (default 24 hours). The client stores the token and api.js injects it as an Authorization: Bearer header on every request. JwtTokenFilter validates the token on each call and populates the security context, so no server-side session exists and the service tier can scale horizontally.

6.2 Role-Based Access Control

Table 20: Access matrix (derived from SRS user classes and NFR-SEC-1)

Resource	Public	Farmer	Admin
/api/auth/register, /api/auth/login	Yes	Yes	Yes
Crop, weather, fertilizer, profit, calendar, price, mandi tools	Yes	Yes	Yes
/api/farmers/** (own records)	No	Yes	Yes (all)
/api/admin/**, /api/market/prices, /api/advisory	No	No	Yes

6.3 Data Protection and Input Handling

- Passwords are salted and hashed with BCrypt (cost factor 10) and are never stored or logged in plain text (NFR-SEC-2).

- All persistence uses Spring Data JPA parameterised queries and prepared statements, eliminating SQL injection (NFR-SEC-3).
- Dynamic frontend rendering uses textContent or pre-sanitised templates to prevent cross-site scripting (NFR-SEC-4).
- CORS is restricted through CorsConfigurationSource to configured origins, including http://localhost:3000 and https://*.netlify.app.
- Foreign keys with cascading rules prevent orphan records when accounts are removed.

6.4 Agronomic Safety as a Security Property

Safety is treated as a first-class property. Fertilizer recommendations are bounded by ICAR thresholds so that the system cannot recommend loads that cause soil toxicity, nitrogen burn or groundwater pollution (NFR-SAFE-1), and no weather or price value is ever randomised or fabricated (NFR-SAFE-2).

Figure 30: SecurityConfig and JwtTokenFilter

7 Testing And Verification

7.1 Testing Strategy

Verification is multi-level and automated where possible. Unit and integration tests are written with JUnit 5 and Spring MockMvc and executed with mvn test; they validate context initialisation, algorithm engines, security filters and repository transactions. Client-side scripts are checked for syntax with node -c and for error-boundary behaviour when backend endpoints are missing. Agronomic boundary verification compares the fertilizer outputs with ICAR benchmarks and checks the acidic (pH < 6.0) and alkaline (pH > 8.0) amendment rules.

7.2 Automated Test Cases

Table 21: Named test cases and the behaviour they verify (from the SRS traceability matrix)

Test case	Module	Verifies
Context load and auth integration tests	M1	Application starts; unauthenticated users cannot reach /api/farmers/ routes
Admin integration tests	M9	Unauthenticated users cannot reach /api/admin/ routes
testRiceRecommendation	M2	Rule match: high rainfall, clay soil and high nitrogen prescribes rice
testWeatherAndIrrigationService	M3	Live weather retrieval and irrigation decision
testWeatherMultipleLocations	M3	Geocoding and forecast for several locations; 404 and 5,000 ms time-out handling
testFertilizerRecommendationBalanced	M4	Correct deficits and bag conversions for a balanced case
testFertilizerAcidicAndAlkalineSoilWarnings	M4	Lime for pH < 6.0 and gypsum for pH > 8.0; no unsafe load
testProfitCalculatorEconomics	M5	Revenue, net profit, ROI and break-even calculation

Test case	Module	Verifies
testCropCalendarStagesAndStateAdvisory	M6	Six stages and state-specific notice
testPricePredictionSuccessWithHistoricalRecords	M7	Forecast, trend and confidence with enough data
testPricePredictionInsufficientDataForRareFilter	M7	INSUFFICIENT_DATA response with fewer than three points
testNearbyMandiPricesJaipur	M8	Nearby results and transparent cached-data tag on API failure
testNearbyMandiRadiusFiltering	M8	Mandis beyond the radius excluded

In total 16 core test cases were executed with 0 failures and 0 errors. The remaining cases are the context-load and authentication / administration integration tests that are not individually named in the SRS.

Figure 31: Maven test run summary (16 tests, 0 failures, 0 errors)

Figure 32: Example JUnit 5 / MockMvc test class

7.3 Non-Functional Verification Status

Table 22: Status of non-functional requirements

Requirement	Target	Verification status
Backend response time	250 ms	Design target; formal load benchmark to be recorded
External API time-out	5,000 ms	Verified by integration test (graceful 404)
Page load	< 1.5 s on 4G	Design target; Lighthouse / DevTools measurement to be recorded
Zero fake data	No fabricated values	Verified by design and failover tests
Fertilizer safety	ICAR thresholds	Verified by acidic / alkaline boundary tests
Security	JWT, BCrypt, JPA, XSS	Verified for access control; others by design review

The report states the performance figures as targets because measured benchmark results were not available in the project record. Recording measured response times (for example from browser DevTools or a load tool) would strengthen this section.

7.4 Requirements Traceability

Appendix A reproduces the Requirements Traceability Matrix linking each requirement group to its backend component, frontend component and test case. Every functional requirement group has at least one verifying test or integration check.

8 Deployment And Confriguration

8.1 Environments

Table 23: Development and production environments

Aspect	Development	Production
Database	H2 in-memory, MODE=MySQL	Cloud MySQL 8.0+ with InnoDB, utf8mb4
Backend	Spring Boot run locally on Java 17	Docker container with the executable JAR on Render
Frontend	Static files served locally (e.g. http://localhost:3000)	Netlify static hosting
CORS	localhost origins	https://*.netlify.app

8.2 Containerisation

The final milestone finalised the Docker configuration for the Spring Boot executable JAR, so the backend is portable to any OCI-compliant container runner. The frontend has no build step and is deployed as static assets. The design targets an availability of 99.5% on the cloud deployment.

Figure 33: Dockerfile and application configuration

8.3 Configuration of External Services

Open-Meteo requires no API key for standard queries. The data.gov.in AGMARKNET API requires an api-key; if it is missing or the service is unreachable, the mandi service falls back to the verified database and discloses this through the source tag. Secrets such as the key and the JWT signing secret should be supplied through environment variables on Render rather than committed to the repository.

9 Project Management

9.1 Team Structure

Table 24: Team roles

Member	Role	Responsibility
Ravina	Team Leader and Tech Lead	Identity, profiling and administration (Modules 1 and 9); security integration tests
Yashvardhan Singhal	Core Backend Developer	Crop recommendation and weather engines (Modules 2 and 3); API latency and fallback tests
Vinay Choudhary	Core Backend Developer	Fertilizer advisor and profit/ROI engine (Modules 4 and 5); agronomic boundary audits
Venkatesh Kumawat	Frontend Developer	Crop calendar, price prediction and mandi locator (Modules 6, 7, 8)
Shrashti Jain	Frontend Developer and QA Lead	Modules 6, 7, 8; data failover and routing validation

9.2 Weekly Progress Record

Table 25: Ten-week PBL timeline

Week	Focus	Outcome
Week 2	User roles, architecture and core modules	Roles (farmer, admin, public), stateless JWT RBAC, nine-module architecture, database strategy and NFRs defined
Week 3	UML design	Class diagram (FarmerController, CropRecommendationService, MandiLocationRegistry) and architecture of asynchronous JSON Fetch calls
Week 4	Database design and UI mock-ups	Six-table schema with referential integrity; Agro Forest Green UI mock-ups
Week 5	Project initialisation and identity engine	Directory structure; secured login, JWT generation, multi-plot UI and admin dashboard (Modules 1 and 9)
Week 6	Agronomy and telemetry engines	Crop prescription endpoint; Open-Meteo integration; smart irrigation advisory (Modules 2 and 3)
Week 7	Financial and fertilizer calculators	ICAR fertilizer calculator with bag conversion; ROI engine and financial classifier (Modules 4 and 5)
Week 8	Calendars and mandi locator	Crop calendar, price prediction and GPS mandi locator with fallback (Modules 6, 7, 8)
Week 9	Integration and E2E testing	16 core test cases executed with 0 failures and 0 errors
Week 10	Final evaluation and submission	SRS v2.0, presentation, video demonstration, Docker configuration, individual demo and viva

9.3 Deliverables

Table 26: Project deliverables

Deliverable	Description
Final project report	Complete SRS v2.0 (nine modules, NFRs, traceability matrix) and this project report
Presentation deck	Architecture, Haversine formula usage and failover states
Video demonstration	Farmer login, live weather fetch and mandi navigation
Production deployment	Cloud-ready: Netlify (frontend) and Render (Java backend)
Source code	https://github.com/imyash058-hub/krishi-management-system

10 Result And Decision

10.1 Achievement of Objectives

Table 27: Objectives versus outcomes

Objective	Outcome
Secure farmer management	JWT RBAC, BCrypt, multi-plot profiles, cascading deletes; unauthenticated access to protected routes rejected in tests
Crop recommendation	Eight-parameter rule-based engine for ten crops with persisted history

Objective	Outcome
Live weather and irrigation	Open-Meteo integration with explicit PAUSE / HEAVY / MODERATE / LIGHT decisions and honest failure handling
Fertilizer advice	Deficits converted to 50 kg and 25 kg bags with lime / gypsum rules and cost estimate
Profit analysis	Seven-cost aggregation, ROI, break-even and five-level classification
Calendar and price trend	Six-stage calendar with state notices; moving-average and momentum forecast with confidence
Mandi locator	Haversine search with radius filter, sorting, navigation links and source transparency
Administration	Statistics, farmer audit, price seeding and scheme publication
Verification	16 test cases, 0 failures, 0 errors

10.2 Comparison with Existing Work

Compared with the surveyed systems (Table 2), Smart Krishi's distinguishing features are integration across the decision chain, actionable outputs (bags, pause/irrigate decisions, break-even prices) and strict data honesty. The literature shows that advisories succeed when they are trusted and actionable [27], [14], and that credibility of forecasts matters for sustained adoption [8]; the transparent source tags and the refusal to simulate data are design responses to these findings.

10.3 Discussion

The rule-based approach makes every output auditable and explainable to a mentor or agronomist, and avoids reliance on training datasets of uncertain provenance. The trade-off is that the system cannot learn from data. The lightweight frontend favours low-bandwidth use but offers less rich interactivity than a single-page-application framework. The platform has been verified through automated tests but has not yet been evaluated in a field trial with farmers; the evidence from advisory studies shows trust and adoption without systematic yield gains [27], so no claim of improved yield or income is made.

11 Limitation And Future Work

11.1 Limitations

- The price engine uses moving averages and linear momentum, simpler than ARIMA, ARFIMAX or machine-learning models used in the literature [18]-[21].
- Crop recommendation is rule-based over ten crops; published ML systems learn from larger datasets [1]-[4].
- Performance targets (250 ms and 1.5 s) have not been recorded as measured benchmarks.
- The platform depends on third-party APIs (Open-Meteo, data.gov.in); the failover preserves honesty but not completeness of live data.
- No field evaluation with farmers has been carried out; impact on yield or income is unproven.
- The interface is not yet available in regional languages or by voice, which limits reach among low-literacy users.

11.2 Future Work

- Replace or complement the price engine with ARIMA or hybrid models, validated using MAPE and RMSE on AGMARKNET history.
- Build a hybrid crop recommender in which rules guarantee safety and a learned model ranks candidate crops.
- Add regional-language and voice interfaces, SMS or WhatsApp delivery, and offline-tolerant progressive web app features.
- Integrate Soil Health Card data directly so that farmers need not type soil values.
- Conduct a pilot with farmers to measure adoption, input savings and satisfaction.
- Record and publish measured latency and page-load benchmarks, and add continuous integration with automated deployment.

12 Conclusion

Smart Krishi demonstrates that the chain of decisions facing a smallholder farmer, from choosing a crop to finding a market, can be supported by a single, secure and honest platform. The literature survey of 32 studies showed that soil-aware fertilizer advice, forecast-based irrigation, price information and market access each have documented benefits, while adoption barriers call for simple, trustworthy and low-cost tools. The nine-module implementation on Spring Boot, MySQL and a lightweight Bootstrap frontend translates these findings into rule-based, ICAR-aligned outputs expressed in units farmers actually use, secured by stateless JWT role-based access control and protected by a failover design that never fabricates data. Sixteen automated test cases passed with no failures or errors.

The next steps are to validate the platform with real farmers, strengthen forecasting with statistical and machine-learning models, and extend access through regional languages. The project also gave the team practical experience in requirements engineering, secure API design, external API integration, relational modelling, testing and cloud deployment.

13 References

[1] S. Kathoon, N. V. Naik, T. V. Kumar, K. Narasimha, A. V. Srinivasulu, E. H. S. N. Goud, "Crop Recommendation System Using Machine Learning," Proc. ICRDICCT'25, SciTePress, pp. 5–9, 2025. https://scitepress.org/PublishedPapers/2025/138908

[2] "AI-based Smart Crop Recommendation System for Sustainable Agricultural Production," Madras Agricultural Journal, 2025. https://www.masujournal.org/view_journal.php?id=566

[3] "A Machine Learning-Enabled System for Crop Recommendation," Engineering Proceedings, vol. 67, no. 1, art. 51. (MDPI; confirm authors and DOI on Google Scholar)

[4] "A Reliable Machine Learning Framework for Crop Recommendation in Precision Agriculture," Genetics and Molecular Research. https://geneticsmr.com/index.php/gmr/article/download/1814/1108/5297

[5] J. M. Miras-Ávalos, J. S. Rubio-Asensio, J. M. Ramírez-Cuesta, J. F. Maestre-Valero, D. S. Intrigliolo, "Irrigation-Advisor—A Decision Support System for Irrigation of Vegetable Crops," MDPI, 2019/2020. https://repositorio.upct.es/handle/10317/9384

[6] A. Pelosi et al., "Predicting crop evapotranspiration by integrating ground and remote sensors with air temperature forecasts." https://www.iris.unisa.it/handle/11386/4738031

[7] "Validation of medium range weather forecasts and its economic impact on …," Punjab Agricultural University, Ludhiana. https://metnet.imd.gov.in/docs/mausamdocs/173414_F.pdf

[8] "Exploring farmers' communication pattern and satisfaction regarding the adoption of Agromet advisory services in semi-arid regions of southern India," Front. Sustain. Food Syst., 2023. https://doi.org/10.3389/fsufs.2023.1284880

[9] G. Chander, P. Choudhari, G. Sawargaonkar, A. Mishra, R. K. Nayak, "Developing Soil Test-based Fertilizer Recommendations," in Mapping the Nutrient Status of Odisha's Soils, ICRISAT, pp. 1–49, 2020. http://oar.icrisat.org/id/eprint/11940

[10] "User-centered design of a fertilizer recommendation system for smallholder farmers," MIT thesis (advisor C. Vaishnav). https://dspace.mit.edu/handle/1721.1/122090

[11] M. K. Sharma et al., "FertiCal-P: An Android-based Decision Support System Determining the NPK Fertilizer Recommendation by Assessing Soil pH," Gujarat Technological University. https://www.agriculturejournal.org/download/24713

[12] "Enhancing Nutrient Use Efficiency in Cowpea through Soil Test Crop Response and Decision-Support Tools in Karnataka," Madras Agricultural Journal, 2025. https://masujournal.org/112/MAJ-2025-1231_1759989448.pdf

[13] "Decision support system for fertilizer recommendation – A case study," Indian Society of Agronomy, 2014. https://pub.isa-india.in/index.php/ija/article/download/4563/4238

[14] "The soil health card scheme in India: lessons learned and challenges for replication in other developing countries," ICAR KRISHI repository. https://krishi.icar.gov.in/jspui/handle/123456789/76131

[15] "Impact of Soil Health Card on Urea Fertilizer Usage and Crop Yield in Haryana," Indian J. Extension Education. https://epubs.icar.org.in/index.php/IJEE/article/download/165252/59933/461118

[16] "Socio-economic impact of Soil Health Card scheme in the state of Andhra Pradesh," Indian J. Agricultural Sciences. https://epubs.icar.org.in/index.php/IJAgS/article/view/134765

[17] "Profitability of Coarse Cereals Production in India." https://yadda.icm.edu.pl/yadda/element/bwmeta1.element.agro-1bb01e5b-1bfe-44cd-b578-5f5e15389760/c/ILNS.39.10.pdf

[18] A. G. A. Cariappa, B. Kathayat, S. Karthiga, R. Sendhil, "Price analysis and forecasting for decision making: Insights from wheat markets in India," Indian J. Agric. Sci., vol. 90, no. 5, 2020. https://doi.org/10.56093/ijas.v90i5.104376

[19] A. Kumar, V. Kumar, Chetna, S. Ghalawat et al., "Forecasting cotton prices in major Haryana markets: A time series and ARIMA approach," Indian J. Agric. Sci., vol. 94, no. 9, 2024. https://doi.org/10.56093/ijas.v94i9.150524

[20] R. K. Paul, D. Mitra, H. S. Roy, A. K. Paul, M. D. Yeasin, "Forecasting price of Indian mustard using long memory time series model incorporating exogenous variable," Indian J. Agric. Sci., vol. 92, no. 7, 2022. https://doi.org/10.56093/ijas.v92i7.103633

[21] H. P. Suresha, M. Uppaladinni, "Indian Commodity Market Price Comparative Study of Forecasting Methods – A Case Study on Onion, Potato and Tomato," Asian J. Research in Computer Science, vol. 12, no. 4, 2021. https://race.reva.edu.in/?p=12997

[22] "An interpretable produce price forecasting system for small and marginal farmers in India using collaborative filtering and adaptive nearest neighbors."

https://research.polyu.edu.hk/en/publications/an-interpretable-produce-price-forecasting-system-for-small-and-m/

[23] D. S. Negi, P. S. Birthal et al., "Farmers' choice of market channels and producer prices in India: Role of transportation and communication networks," Food Policy. https://digitalcommons.isical.ac.in/journal-articles/1129

[24] P. Venkatesh et al., "The changing structure of agricultural marketing in India: …" (e-NAM study). https://ageconsearch.umn.edu/record/320462/files/08-P%20Venkatesh.pdf

[25] J. R. Sai Shiva, S. R. Nair, T. Thajudeen, "Assessing the Impact of Agriculture Market Digitization on Farmers Price Realization: The Case of Indian eNAM," IIM Kozhikode Working Paper 491. https://econpapers.repec.org/RePEc:iik:wpaper:491

[26] "Agricultural marketing channels and market prices: evidence from …," Economic Analysis and Policy, vol. 81, pp. 1308–1321, 2024. https://asu.elsevierpure.com/en/publications/agricultural-marketing-channels-and-market-prices-evidence-from-h

[27] S. Cole, A. N. Fernando, "'Mobile'izing Agricultural Advice: Technology Adoption, Diffusion and Sustainability," Economic Journal, vol. 131, no. 633, pp. 192–219, 2021. https://doi.org/10.1093/ej/ueaa084

[28] T. Harigaya, J. Zhu, "Customized Agricultural Advisory Services for Smallholder Farmers in India," Precision Development report, 2025. https://precisiondev.org/wp-content/uploads/2025/03/Customized-Advisory-Services-in-India-PxD-March-2025.pdf

[29] M. Shaheen, M. K. Soma, F. Zeba, M. Aruna, "Precision agriculture in India – challenges and opportunities," Int. J. Agricultural Resources, Governance and Ecology, vol. 16, no. 3/4, 2020. https://inderscience.com/offers.php?id=115331

[30] "Whither digital agriculture in India?" (peer-reviewed review). https://researchportal.murdoch.edu.au/esploro/outputs/journalArticle/Whither-digital-agriculture-in-India/991005593760907891

[31] R. Rajesh, M. Malarkodi, "Role of Precision Agriculture Technologies in Improving Crop Productivity and Farm Profitability in India: A Narrative-Integrative Review." https://geneticsmr.com/index.php/gmr/article/view/4896

[32] C. Sathyamala, "Digitalisation of farm practices and challenges faced by marginal and small farmers in India," Institute for Human Development (conference paper). https://nomadit.co.uk/conference/130/paper/90617

[R1] National Productivity Council, "Soil Testing Infrastructure for Faster Delivery of Soil Health Card in India," 2017 (summary by Govt. of India, PIB): over 90% of surveyed farmers had not tested soil before the scheme; 92% were not using balanced fertilizer.

[R2] CEGA, UC Berkeley, "Alleviating Constraints to Adoption of Soil Management": most Indian farmers rely on blanket fertilizer recommendations that ignore current soil fertility. https://cega.berkeley.edu/research/alleviating-constraints-to-adoption-of-soil-management

[R3] Open-Meteo, "Reference Evapotranspiration (ET0)" documentation: hourly/daily ET0 forecasts, free for non-commercial use without an API key. https://openmeteo.substack.com/p/reference-evapotranspiration-for

[R4] Smart Krishi Software Requirements Specification, version 2.0 (October 2026), prepared per IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018.

[S1] Open-Meteo documentation: https://open-meteo.com/en/docs

[S2] ICAR, Handbook of Agriculture and Soil Nutrient Management guidelines

[S3] Spring Boot Reference Guide (v3.3.4) and Spring Security 6 documentation

[S4] National Agriculture Market (e-NAM) and AGMARKNET / data.gov.in, Ministry of Agriculture and Farmers Welfare

Appendix A: Screenshot and Code Capture Checklist

```java
package com.smartkrishi.repository;

import com.smartkrishi.entity.Advisory;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface AdvisoryRepository extends JpaRepository<Advisory, Long> {
    List<Advisory> findByCategoryIgnoreCase(String category);
    List<Advisory> findBySeasonIgnoreCase(String season);
    List<Advisory> findTop4ByOrderByCreatedAtDesc();
}
```

```java
package com.smartkrishi.repository;

import com.smartkrishi.entity.CropRecommendationRecord;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface CropRecommendationRepository extends JpaRepository<CropRecommen
    List<CropRecommendationRecord> findByUserIdOrderByCreatedAtDesc(Long userId)
    List<CropRecommendationRecord> findTop5ByOrderByCreatedAtDesc();
}
```

```java
package com.smartkrishi.controller;

import com.smartkrishi.dto.ApiResponse;
import com.smartkrishi.dto.PricePredictionResponse;
import com.smartkrishi.service.PricePredictionService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/price-prediction")
public class PricePredictionController {

    private final PricePredictionService pricePredictionService;

    public PricePredictionController(PricePredictionService pricePredicti
        this.pricePredictionService = pricePredictionService;
    }

    /**
     * Statistical baseline price prediction & market momentum analysis.
     * GET /api/price-prediction/{crop}?state={state}&district={district}
     */
    @GetMapping("/{crop}")
    public ResponseEntity<ApiResponse<PricePredictionResponse>> getPriceP
            @PathVariable String crop,
            @RequestParam(required = false) String state,
            @RequestParam(required = false) String district,
            @RequestParam(required = false, defaultValue = "10") Integer

        PricePredictionResponse response = pricePredictionService.predict
        return ResponseEntity.ok(ApiResponse.ok("Market price trend and p
    }
}
```

```bash
INPUT=$(cat)

TOOL_NAME="${INPUT#*\"tool_name\":\"}"
TOOL_NAME="${TOOL_NAME%%\"*}"

case "$TOOL_NAME" in
  run_in_terminal|appmod-*) ;;
  *) exit 0 ;;
esac

case "$INPUT" in
  *'"session_id":"'*) ;;
  *) exit 0 ;;
esac

SESSION_ID="${INPUT#*\"session_id\":\"}"
SESSION_ID="${SESSION_ID%%\"*}"
[ -z "$SESSION_ID" ] && exit 0

HOOKS_DIR=".github/modernize/java-upgrade/hooks"
mkdir -p "$HOOKS_DIR"

LINE=$(printf '%s' "$INPUT" | tr -d '\r\n')
printf '%s\n' "$LINE" >> "$HOOKS_DIR/${SESSION_ID}.json"
```